UNIVERSIDAD NACIONAL DE SAN ANTONIO ABAD DEL CUSCO

**FACULTAD DE INGENIERÍA ELÉCTRICA,
ELECTRÓNICA, INFORMÁTICA Y MECÁNICA**

**ESCUELA PROFESIONAL DE INGENIERÍA INFORMÁTICA
Y DE SISTEMAS**



INFORME TÉCNICO DE INVESTIGACIÓN

"RUTAS MÍNIMAS CON DISTINTAS COLAS DE PRIORIDAD EN REDES VIALES URBANAS DE LA CIUDAD DEL CUSCO"

Asignatura: ALGORITMOS AVANZADOS (Semestre 2026-2)

Docente Responsable: Ing. Héctor Eduardo Ugarte Rojas

Identificador: GRUPO 5

Presentado por:

- Choquenaira Quispe, Noe Franklin 133962
- Porroa Sivana, Yeni Ruth 120893
- Quispe Rimachi, Romario 164257
- Yaranga Achahui, Aldo 103179

Cusco – Perú – 2026

Índice

1. Introducción **1**
1.1. Contexto y Motivación . . . 1
1.2. Planteamiento del Problema . . . 1
1.3. Objetivos del Proyecto . . . 2
1.3.1. Objetivo General . . . 2
1.3.2. Objetivos Específicos . . . 2
1.4. Alcance y Delimitación . . . 2
1.5. Control de Versiones y Repositorio Oficial en GitHub . . . 3
1.6. Organización del Documento . . . 3

2. Fundamentos Teóricos y Aprendizaje Autónomo **4**
2.1. Conceptos y Algoritmos Requeridos . . . 4
2.1.1. Grafos Viales Urbanos y Propiedad de Red Plana . . . 4
2.1.2. Algoritmo de Dijkstra e Invariante de Subestructura Óptima . . . 4
2.2. Fuentes Académicas Estudiadas y Criterios de Selección . . . 4
2.3. Diagnóstico Formal de Brechas de Conocimiento y Retos Técnicos . . . 5
2.4. Plan Individual y Grupal de Aprendizaje y Especialización . . . 6
2.5. Estrategia y Resolución de Dificultades Técnicas . . . 6

3. Evaluación y Selección del Stack Tecnológico **8**
3.1. Identificación de Herramientas Candidatas . . . 8
3.2. Matriz Multicriterio de Comparación Tecnológica . . . 8
3.3. Tecnología Seleccionada y Justificación Técnica . . . 9
3.4. Procedimiento de Aprendizaje e Integración del Stack . . . 9
3.5. Registro de Dificultades de Adopción y Soluciones Aplicadas . . . 10

4. Formulación Matemática del Problema y Caso de Estudio en Cusco **11**
4.1. Definición Formal del Grafo Vial Urbano . . . 11
4.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema . . . 11
4.2.1. Parámetros de Entrada . . . 11
4.2.2. Variables de Decisión Binarias . . . 11
4.2.3. Salidas del Sistema . . . 12
4.3. Restricciones de Conservación de Flujo y No Negatividad . . . 12
4.4. Función Objetivo y Modelo de Costo Generalizado para Cusco . . . 12
4.4.1. Formulación de la Impedancia Vial Andina . . . 12
4.4.2. Dijkstra Modificado por Impedancia Topográfica Andina . . . 13
4.5. Instancia Pequeña Resuelta Manualmente: Centro Histórico del Cusco . . . 13
4.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$) . . . 14
4.6.1. Reconstrucción de Rutas Mínimas Óptimas . . . 16

5. Diseño Algorítmico y Estructuras de Colas de Prioridad **17**
5.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta . . . 17
5.2. Invariantes de Bucle y Prueba de Corrección . . . 17

5.3. Estructuras de Datos de Cola de Prioridad Evaluadas . . . 18
5.3.1. Min-Heap Binario Indexado (`IndexedMinHeap`) . . . 18
5.3.2. 4-ary Heap Contiguo (`DAryHeap`) . . . 19
5.3.3. Fibonacci Heap (`FibonacciHeap`) . . . 19
5.3.4. Pairing Heap (Perspectiva Teórica) . . . 19
5.4. Matriz Comparativa de Complejidad Asintótica . . . 20

6. Diseño e Implementación del Sistema **21**
6.1. Arquitectura Modular del Software . . . 21
6.2. Descripción de Módulos del Sistema . . . 21
6.3. Visualización de Grafos Viales con Graphviz (.dot) . . . 22
6.4. Rol de NetworkX como Oráculo de Verificación Canónica . . . 23
6.5. Formatos de Datos y Serialización JSON . . . 23
6.6. Contratos de Interfaz de las Estructuras de Datos . . . 23
6.7. Implementación de Algoritmos y Estructuras Nucleares . . . 24

7. Metodología Experimental y Diseño del Banco de Pruebas **31**
7.1. Preguntas de Investigación Científica . . . 31
7.2. Formulación de Hipótesis Técnicas . . . 31
7.3. Variables Experimentales . . . 31
7.4. Métricas de Evaluación e Instrumentación de Precisión . . . 32
7.5. Entorno de Ejecución y Protocolo Reproducible . . . 32

8. Resultados y Validación Computacional del Prototipo Inicial **33**
8.1. Verificación de Distancias y Rutas Mínimas en la Red de Cusco . . . 33
8.2. Análisis Cuantitativo de Operaciones Elementales en Colas . . . 33
8.3. Comparativa Preliminar de Rendimiento Temporal en Instancia Base . . . 34
8.4. Gráficos Comparativos y Cartografía Generada . . . 34
8.5. Protocolo de Demostración sobre el Sistema sin Diapositivas . . . 36
8.6. Dificultades Técnicas Identificadas y Ajustes Realizados . . . 36

9. Estrategia de Pruebas y Validación Automatizada **37**
9.1. Framework de Pruebas y Patrón Arrange-Act-Assert (AAA) . . . 37
9.2. Tipología y Cobertura de Casos de Prueba . . . 37
9.3. Validación Cruzada y Equivalencia Algorítmica . . . 38

10. Avance 4: Implementación de la Primera Alternativa **39**
10.1. Flujo de Ejecución sobre el Sistema . . . 39
10.2. Decisiones de Implementación y Estructuras de Datos . . . 39
10.3. Correspondencia entre Pseudocódigo y Código Fuente . . . 40
10.4. Salida Verificable: Certificado de Optimalidad . . . 41
10.5. Validación con Instancias Resueltas Manualmente . . . 42
10.6. Pruebas Automatizadas . . . 42
10.7. Modificaciones Respecto del Diseño Inicial . . . 43
10.8. Pendientes, Errores Conocidos y Dificultades . . . 43

11. Avance 5: Segunda Alternativa e Integración del Sistema **44**
11.1. Interfaz Común e Integración de Componentes 44
11.2. Manejo de Entradas Inválidas 45
11.3. Equivalencia de Resultados y Explicación de Diferencias 45
11.4. Resultados de Escalabilidad 46
11.5. Pruebas de Integración y Equivalencia 46
11.6. Pendientes, Errores Conocidos y Dificultades 47

Anexos **48**
Anexo A: Manual de Instalación y Configuración 48
Anexo B: Manual de Uso y Ejecución 48
Anexo C: Bitácora de Aprendizaje Autónomo 50
Anexo D: Matriz de Trazabilidad de Objetivos 51
Anexo E: Evidencias de Seguimiento 51

Referencias Bibliográficas **52**

Índice de figuras

1. Grafo vial dirigido representativo del Centro Histórico del Cusco ($|V| = 6, |E| = 9$) . . . 14
2. Diagrama arquitectural desacoplado del sistema en cuatro capas operativas 21
3. Visualización cartográfica y topológica de la red vial del Centro Histórico del Cusco generada por el prototipo 35
4. Comparativa entre distancia física y tiempo de viaje con impedancia topográfica en Cusco 35

Índice de tablas

1. Selección razonada de fuentes bibliográficas clave y criterios de pertinencia 5
2. Plan de aprendizaje y distribución de líneas de especialización (Grupo 5) 6
3. Herramientas candidatas evaluadas por capa tecnológica 8
4. Matriz multicriterio para la selección del lenguaje y entorno de desarrollo (Escala 1 a 5) . 9
5. Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco. 14
6. Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo. . . 16
7. Matriz comparativa de complejidades temporales y espaciales de colas de prioridad en Dijkstra 20
8. Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor 33
9. Tiempos medios de ejecución del algoritmo de Dijkstra según la cola de prioridad (instancia $N = 6$) 34
10. Clasificación de la suite de pruebas unitarias y casos evaluados 38
11. Estructuras de datos de la primera alternativa 40
12. Correspondencia entre el Algoritmo 1 y la implementación 40
13. Pruebas automatizadas de la primera alternativa 42
14. Módulos reutilizados por todas las alternativas 44
15. Clasificación de diferencias entre alternativas 45
16. Tiempo medio (ms) de Dijkstra por cola sobre redes viales sintéticas (7 pasadas) 46
17. Bitácora de autoaprendizaje individual y resolución de dificultades 50
18. Matriz de correspondencia entre objetivos y secciones del informe 51

1. Introducción

1.1. Contexto y Motivación

La determinación eficiente de rutas mínimas en redes viales urbanas representa uno de los problemas fundamentales y de mayor impacto práctico en la ciencia de la computación, la ingeniería de transporte y la logística de atención a emergencias. En entornos metropolitanos contemporáneos, los sistemas de despacho para ambulancias, unidades de bomberos y patrullaje policial requieren calcular trayectorias óptimas en fracciones de segundo para minimizar los tiempos de respuesta, donde reducciones marginales de tiempo se traducen directamente en la preservación de vidas humanas.

La ciudad del Cusco, ubicada en la sierra sur del Perú a una altitud media de 3,399 m s. n. m., presenta una morfología urbana singular. Su Centro Histórico, declarado Patrimonio de la Humanidad por la UNESCO en 1983, superpone una traza urbana incaica y colonial caracterizada por calzadas sumamente angostas, pasajes adoquinados, alta densidad de giros obligatorios y severas pendientes topográficas que oscilan con frecuencia entre el 5 % y más del 15 %. En este contexto geográfico, la distancia euclidiana o la longitud puramente física de un tramo vial resultan insuficientes como métrica de optimización. Un trayecto físicamente más corto pero con un gradiente positivo pronunciado impone un esfuerzo motriz severo sobre un vehículo pesado de emergencia, reduciendo drásticamente su velocidad operativa y aumentando el riesgo de demoras críticas. Por ende, resulta imperativo formular modelos de costo generalizado que integren la topografía andina y evaluar algoritmos de caminos mínimos capaces de procesar mallas viales a gran escala con alta eficiencia temporal y espacial.

1.2. Planteamiento del Problema

El Algoritmo de Dijkstra (Dijkstra, 1959) es el estándar canónico para resolver el problema de caminos mínimos desde un origen único (*Single-Source Shortest Path*, SSSP) en grafos dirigidos con pesos no negativos ($w(e) \geq 0$). La complejidad asintótica de este algoritmo depende intrínsecamente de la estructura de datos utilizada para implementar la cola de prioridad que gestiona los vértices tentativos:

$$T(|V|,|E|) = O(|V| \cdot T_{\text{Extract-Min}} + |E| \cdot T_{\text{Decrease-Key}} + |V| \cdot T_{\text{Insert}}) \tag{1}$$

En la literatura teórica clásica, el Fibonacci Heap propuesto por Fredman y Tarjan (1987) ostenta la cota asintótica óptima conocida de $O(|E| + |V| \log |V|)$, al ejecutar la operación de disminución de clave (`Decrease-Key`) en tiempo amortizado $O(1)$ y la extracción del mínimo (`Extract-Min`) en $O(\log |V|)$. No obstante, estudios computacionales de referencia (Cherkassky et al., 1996; LaMarca & Ladner, 1996) han reportado que, en redes dispersas donde $|E| = O(|V|)$ (como las redes viales donde el grado promedio por nodo rara vez supera 4), la constante oculta en la notación asintótica del Fibonacci Heap, originada por la alocación dinámica en memoria, la manipulación de múltiples punteros y los fallos en las líneas de caché (*cache misses*), degrada su rendimiento en tiempo real frente a montículos contiguos como el Binary Heap o el d-ary Heap.

El problema central que aborda este proyecto radica en investigar, implementar y contrastar cuantitativamente el comportamiento empírico y teórico de distintas colas de prioridad acopladas al algoritmo de Dijkstra, evaluando su desempeño real sobre la red vial urbana del Cusco bajo una función

de costo generalizado con impedancia topográfica.

1.3. Objetivos del Proyecto

1.3.1. Objetivo General

Diseñar, implementar y evaluar experimentalmente una solución computacional modular para el cálculo de rutas mínimas en la red vial urbana de la ciudad del Cusco, contrastando el desempeño en tiempo de ejecución, operaciones elementales y consumo de memoria del Algoritmo de Dijkstra instrumentado con tres estructuras de colas de prioridad: Min-Heap Binario Indexado, d-ary Heap y Fibonacci Heap.

1.3.2. Objetivos Específicos

Siguiendo la metodología de ingeniería de software y análisis de algoritmos, los objetivos específicos se ordenan en cuatro fases secuenciales e integradas:

1. **OE1 (Modelar):** Formular el modelo matemático del grafo vial dirigido del Centro Histórico del Cusco, definiendo una función de costo generalizado (impedancia andina) que cuantifique el impacto de calles de un solo sentido, calles angostas coloniales, pasajes peatonales y gradientes altitudinales positivos.

2. **OE2 (Diseñar e Implementar):** Diseñar e implementar bajo una arquitectura desacoplada en cuatro capas lógicas el algoritmo de Dijkstra parametrizado, integrando tres alternativas de colas de prioridad puras con soporte formal para `Decrease-Key`: Min-Heap Binario Indexado, 4-ary Heap ($d = 4$) optimizado para líneas de caché L1/L2, y Fibonacci Heap amortizado.

3. **OE3 (Verificar):** Validar analíticamente la correctitud del algoritmo mediante traza manual paso a paso, certificado formal de optimalidad independiente en $O(|V| + |E|)$ y una suite automatizada de pruebas unitarias bajo el patrón Arrange-Act-Assert (AAA), contrastando los resultados contra oráculos canónicos de referencia (NetworkX y Bellman-Ford).

4. **OE4 (Evaluar e Investigar):** Evaluar experimentalmente la escalabilidad asintótica, el consumo de operaciones elementales y los tiempos de CPU en redes viales sintéticas y reales, contrastando la teoría asintótica clásica frente a la localidad espacial de memoria caché en hardware contemporáneo.

1.4. Alcance y Delimitación

El proyecto se delimita formalmente en las siguientes dimensiones:

- **Delimitación Espacial:** La zona de estudio experimental corresponde a la malla vial nuclear del Centro Histórico del Cusco (6 intersecciones estratégicas: Plaza Mayor, Plaza Regocijo, Qorikancha, Plaza San Francisco, Mercado Central San Pedro y Plazoleta Limacpampa). Esta zona constituye el caso de estudio ideal debido a su altísima densidad de pasajes coloniales estrechos, giros de un solo sentido y severas pendientes topográficas. El sistema queda arquitectónicamente

desacoplado y preparado para escalar a la red metropolitana completa mediante ingesta vectorial de OpenStreetMap vía OSMnx.

- **Delimitación Algorítmica:** Se evalúa el algoritmo de Dijkstra en grafos dirigidos estáticos con costos deterministas no negativos ($w(e) \geq 0$). Para contrastar criterios con pesos negativos, se integra Bellman-Ford como algoritmo de referencia y verificación. No se modela congestión estocástica en tiempo real en esta fase.
- **Alcance del Sistema:** Comprende la formulación matemática integral, la solución manual paso a paso, el núcleo ejecutable modular en Python con CLI interactiva, exportación vectorial de grafos con Graphviz (código DOT y diagramas), el cuaderno interactivo en Google Colab y la suite automatizada de 219 pruebas con Pytest.

1.5. Control de Versiones y Repositorio Oficial en GitHub

Conforme a las directrices de reproducibilidad y trabajo colaborativo del curso, el desarrollo íntegro del proyecto se gestiona bajo el sistema de control de versiones Git y se aloja en el repositorio público oficial:

https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC

La organización de ramas adopta un esquema de integración continua estructurado: la rama `main` almacena las versiones estables consolidadas, complementada por ramas de desarrollo temáticas por integrante (`ramaFRANKI`, `ramaAldo`, `ramaYeni`, `RamaRomario`). Los avances computacionales, los datos procesados, la documentación técnica y las pruebas unitarias cuentan con trazabilidad completa mediante commits atómicos y descriptivos.

1.6. Organización del Documento

El presente informe se estructura rigurosamente en las siguientes secciones temáticas: La Sección 2 expone los fundamentos teóricos y el diagnóstico de retos técnicos. La Sección 3 evalúa y justifica el stack tecnológico adoptado mediante una matriz multicriterio. La Sección 4 presenta la formulación matemática y el modelo de costo cusqueño. La Sección 5 detalla el diseño algorítmico, pseudocódigos, invariantes y complejidades. La Sección 6 describe la arquitectura del software y módulos implementados. La Sección 9 especifica la estrategia de pruebas unitarias (AAA). La Sección 7 formula las hipótesis y el diseño del banco de pruebas experimental. La Sección 8 documenta los resultados computacionales del prototipo inicial, la verificación analítica sobre Cusco y el protocolo de demostración en el sistema. Finalmente, las referencias en formato APA 7ma edición y los Anexos técnicos (manuales de instalación y ejecución, bitácora de aprendizaje y matriz de trazabilidad) complementan el informe.

2. Fundamentos Teóricos y Aprendizaje Autónomo

2.1. Conceptos y Algoritmos Requeridos

El desarrollo del proyecto descansa sobre tres pilares conceptuales de las ciencias de la computación: la teoría de grafos viales dirigidos, el diseño de colas de prioridad con clave disminuible y la relajación voraz (*greedy*) de caminos mínimos.

2.1.1. Grafos Viales Urbanos y Propiedad de Red Plana

Una red vial urbana se modela formalmente como un grafo dirigido ponderado $G = (V, E, W)$, donde V representa el conjunto de intersecciones viales o discontinuidades geométricas, $E \subseteq V \times V$ representa los tramos viales orientados según el sentido de circulación reglamentario, y $W : E \to \mathbb{R}^+$ asigna un costo no negativo a cada tramo. Las redes viales urbanas presentan propiedades geométricas cuasi-planas con una densidad extremadamente baja:

$$|E| \approx k \cdot |V|, \quad \text{donde } 2.0 \leq k \leq 3.5 \tag{2}$$

Esta dispersión estructural condiciona de forma determinante el comportamiento asintótico de los algoritmos de optimización de rutas.

2.1.2. Algoritmo de Dijkstra e Invariante de Subestructura Óptima

El algoritmo de Dijkstra (Dijkstra, 1959) explora el espacio de búsqueda manteniendo una partición de los vértices en dos conjuntos disjuntos: el conjunto S de vértices cuya distancia mínima desde el origen s ya ha sido permanentemente determinada, y el conjunto $Q = V \setminus S$ de vértices tentativos gestionados en una cola de prioridad. En cada paso iterativo, el algoritmo extrae el vértice $u \in Q$ con la menor distancia tentativa acumulada $d[u]$, lo incorpora a S, y relaja cada arista saliente $(u, v) \in E$:

$$\text{si } d[u] + w(u,v) < d[v] \implies d[v] \leftarrow d[u] + w(u,v), \quad \pi[v] \leftarrow u \tag{3}$$

La actualización del valor $d[v]$ en la cola de prioridad exige ejecutar la operación `Decrease-Key`$(Q, v, d[v])$.

2.2. Fuentes Académicas Estudiadas y Criterios de Selección

La selección de literatura se condujo con criterios de rigor académico, priorizando artículos seminales arbitrados por pares, manuales algorítmicos estándar de la Association for Computing Machinery (ACM) e IEEE, e investigaciones sobre análisis de rendimiento bajo jerarquía de memoria (Tabla 1).

Tabla 1
Selección razonada de fuentes bibliográficas clave y criterios de pertinencia

Fuente Académica	**Tipología y Dominio**	**Criterio de Pertinencia e Impacto en el Proyecto**
Dijkstra (1959)	Artículo Seminal (Numerische Mathematik)	Fundamento teórico del algoritmo SSSP con pesos no negativos y principio de relajación voraz.
Fredman & Tarjan (1987)	Artículo Teórico (Journal of the ACM)	Especificación formal del Fibonacci Heap con tiempos amortizados $O(1)$ para `Decrease-Key`.
Tarjan (1983)	Monografía Clásica (SIAM)	Análisis riguroso del método del potencial para el cálculo de costos amortizados en estructuras de datos.
Cherkassky et al. (1996)	Estudio Empírico (Journal of Algorithms)	Evaluación experimental exhaustiva de algoritmos de caminos mínimos; evidencia de ineficiencias del Fibonacci Heap en grafos dispersos.
LaMarca & Ladner (1996)	Arquitectura y Algoritmos (ACM SODA)	Análisis del impacto de las líneas de caché L1/L2 y fallos de traducción TLB en montículos binarios y d-arios.
Boeing (2017)	Sistemas Geoespaciales (Computers, Env. and Urban Systems)	Metodología de ingestión, validación y corrección topológica de grafos urbanos a partir de OpenStreetMap.
Cormen et al. (2022)	Libro de Texto Avanzado (MIT Press)	Demostración formal de invariantes de bucle, cotas de recurrencia y pseudocódigo estructurado.

2.3. Diagnóstico Formal de Brechas de Conocimiento y Retos Técnicos

Al inicio del proyecto, el equipo diagnosticó cuatro brechas conceptuales y técnicas que trascendían los contenidos estándar de cursos formativos previos:

1. **Brecha 1: Manipulación de Estructuras Arbóreas con Punteros Múltiples.** Dominio de la representación en memoria de listas circulares doblemente enlazadas con punteros padre, hijo, izquierdo y derecho (*parent, child, left, right*), y ejecución correcta de cortes en cascada (*cascading cuts*) sin fugas de memoria ni bucles infinitos.

2. **Brecha 2: Mapeo Inverso e Indexación en Colas de Prioridad.** Diseño e implementación de tablas de indexación directa ($\text{pos}[v]$) que mantengan sincronizada la posición física de un nodo en un arreglo contiguo frente a operaciones de *sift-up* y *sift-down*, permitiendo `Decrease-Key` en tiempo sublineal $O(\log |V|)$ sin incurrir en búsquedas lineales $O(|V|)$.

3. **Brecha 3: Ingestión y Topología de Grafos Geoespaciales Urbanos.** Extracción, proyección cartográfica y filtrado de componentes conexas a partir de datos vectoriales crudos de OpenStreetMap con OSMnx y GeoPandas.

4. **Brecha 4: Perfilado de Rendimiento a Nivel de Nanosegundos.** Metodología para medir tiempos de CPU reales aislando el ruido del recolector de basura (*Garbage Collector*), efectos de calentamiento de caché (*warm-up*) y lecturas directas de contadores de hardware.

2.4. Plan Individual y Grupal de Aprendizaje y Especialización

Para solventar las brechas identificadas y estructurar el desarrollo, el equipo organizó un plan de trabajo colaborativo e individualizado (Tabla 2).

Tabla 2
Plan de aprendizaje y distribución de líneas de especialización (Grupo 5)

Integrante	Línea de Especialización	Estrategia y Recursos de Estudio
Choquenaira Quispe, Noe Franklin (Líder del Proyecto)	Liderazgo Técnico, Arquitectura del Sistema y Modelado de Impedancia Andina	Concepción de la arquitectura desacoplada en 4 capas, formulación matemática de la impedancia topográfica $\Phi(\theta)$, estructuración geométrica del grafo vial de Cusco, integración del solver de Dijkstra y coordinación técnica del equipo.
Porroa Sivana, Yeni Ruth	Colas de Prioridad Contiguas Indexadas	Análisis del diseño de montículos binarios y d-arios ($d = 4$) con tablas hash inversas y optimización de índices aritméticos en memoria contigua.
Quispe Rimachi, Romario	Estructuras Dinámicas Avanzadas	Estudio de Cormen (Capítulo 19) y Tarjan (1983) para la implementación de listas circulares dobles, árboles enlazados y cortes en cascada en Fibonacci Heap.
Yaranga Achahui, Aldo	Pruebas Automatizadas y Benchmarking	Adopción del estándar Arrange-Act-Assert con Pytest, perfilado con `time.perf_counter_ns` y exportación tipográfica automatizada a LaTeX.

2.5. Estrategia y Resolución de Dificultades Técnicas

Durante la construcción del prototipo, surgieron tres dificultades técnicas sustantivas que fueron analizadas y resueltas por el equipo:

- **Sincronización del Mapeo Inverso en Montículos:** Al implementar el Min-Heap binario, las operaciones de intercambio (*swap*) durante el flotado y hundido desincronizaban la tabla de posiciones pos$[v]$. La solución adoptada consistió en encapsular el intercambio en un método atómico `_swap(i, j)` que actualiza de manera simultánea el arreglo del montículo y las entradas del diccionario inverso.
- **Manejo de Raíces Marcadas en Fibonacci Heap:** La pérdida de un segundo hijo en nodos del Fibonacci Heap requería activar la cascada de cortes hacia los ancestros. Se implementó una rutina recursiva `_cascading_cut` que verifica el estado booleano de la bandera `mark`, restableciéndola a falso al promover el nodo a la lista raíz.
- **Penalización Asimétrica por Pendiente:** La impedancia topográfica en calles cusqueñas no debe penalizar los tramos en descenso de la misma forma que los tramos en ascenso. El modelo se

formuló mediante un operador $\max(0, \text{pendiente})$, asegurando que las bajadas no disminuyan artificialmente el tiempo de viaje por debajo de la velocidad libre de diseño de la vía.

3. Evaluación y Selección del Stack Tecnológico

3.1. Identificación de Herramientas Candidatas

El equipo llevó a cabo un análisis comparativo y riguroso de herramientas emergentes y consolidadas en cuatro capas operativas: lenguaje de programación, manejo de redes viales, framework de pruebas y entorno de documentación (Tabla 3).

Tabla 3
Herramientas candidatas evaluadas por capa tecnológica

Capa	Alternativas Evaluadas	Propósito y Características Evaluadas
Lenguaje Principal	C++20 vs. Rust 1.75 vs. Python 3.12	Rendimiento en microsegundos, sobrecarga de bajo nivel vs. expresividad matemática, tipado y velocidad de prototipado.
Malla Geoespacial	OSMnx / NetworkX vs. Overpass API vs. PostGIS	Descarga automatizada, simplificación topológica, curvas de nivel y cálculo de grafos dirigidos.
Pruebas de Software	Pytest vs. Unittest vs. GoogleTest	Automatización de aserciones, diseño AAA, parametrización de casos límite y cobertura.
Instrumentación	`perf_counter_ns` vs. cProfile vs. Valgrind	Precisión temporal a nivel de nanosegundos, perfilado de llamadas y consumo de memoria.
Documentación	LaTeX (MiKTeX/TeX Live) vs. Typst vs. Word	Formato APA 7ma edición estricto, gestión bibliográfica con Biber/BibLaTeX y ecuaciones avanzadas.

3.2. Matriz Multicriterio de Comparación Tecnológica

Para sustentar la elección bajo criterios cuantitativos y cualitativos objetivos, se formuló una matriz de ponderación multicriterio considerando cuatro variables críticas (Tabla 4):

- **C1: Expresividad y Legibilidad Algorítmica (Peso 30 %):** Capacidad de plasmar las estructuras de datos y algoritmos de forma transparente y directamente verificable contra la literatura matemática.
- **C2: Capacidad de Instrumentación y Aislamiento (Peso 25 %):** Facilidad para medir tiempos de reloj de alta precisión, conteo de primitivas y huella de memoria sin intermediarios intrusivos.
- **C3: Reproducibilidad y Portabilidad Multiplataforma (Peso 25 %):** Capacidad de ejecución inmediata en cualquier sistema operativo (Linux, Windows, macOS) sin requerir cadenas complejas de compilación cruzada.
- **C4: Curva de Aprendizaje y Velocidad de Adopción (Peso 20 %):** Tiempo necesario para que los cuatro integrantes del equipo dominen la tecnología e implementen prototipos tempranos estables y verificables.

Tabla 4
Matriz multicriterio para la selección del lenguaje y entorno de desarrollo (Escala 1 a 5)

Alternativa Evaluada	C1 (30 %)	C2 (25 %)	C3 (25 %)	C4 (20 %)	Puntuación Ponderada
Python 3.12 (Seleccionado)	5.0	4.2	4.8	4.8	**4.71**
C++20 (GCC/Clang)	3.8	4.8	3.5	2.8	**3.78**
Rust 1.75	4.0	4.6	3.6	2.2	**3.69**

3.3. Tecnología Seleccionada y Justificación Técnica

Con base en la evaluación multicriterio, se seleccionó el siguiente stack tecnológico:

1. **Python 3.12+ con anotaciones de tipo (`typing`):** Provee un equilibrio óptimo entre rigurosidad sintáctica y agilidad de experimentación. Las tres estructuras de datos de cola (`IndexedMinHeap`, `DAryHeap`, `FibonacciHeap`) se programaron de forma pura, sin recurrir a paquetes preconstruidos como `heapq` o colas de SciPy, asegurando que la evaluación refleje la lógica algorítmica diseñada.

2. **OSMnx & GeoPandas:** Permiten modelar redes viales reales directamente desde OpenStreetMap, abstrayendo intersecciones como nodos y tramos de calle como aristas dirigidas ponderadas con atributos geométricos y sentidos viales oficiales.

3. **Pytest con formato AAA:** Garantiza la reproducibilidad mediante pruebas unitarias parametrizadas que verifican de manera continua las invariantes de montículo y la exactitud de distancias.

4. **LaTeX con MiKTeX y biblatex-apa:** Plataforma tipográfica que garantiza la adhesión plena al formato APA 7ma edición, generación nativa de tablas científicas `booktabs` e incrustación directa de código fuente con `listings`.

3.4. Procedimiento de Aprendizaje e Integración del Stack

La integración del stack se estructuró en tres fases de adopción temprana:

1. **Entorno Aislado y Gestión de Dependencias:** Configuración de entornos virtuales (`venv`) y definición de un archivo `requirements.txt` libre de rutas absolutas locales, garantizando portabilidad en cualquier máquina de desarrollo.

2. **Construcción del Prototipo de Adopción Temprana:** Desarrollo de la primera versión del Min-Heap indexado acoplado a un solver básico de Dijkstra para 6 intersecciones cusqueñas.

3. **Automatización de Artefactos Tipográficos:** Desarrollo del módulo `latex_exporter.py`, el cual extrae el historial iterativo de Dijkstra y los parámetros de las aristas viales para volcarlos en archivos `.tex` en `documento/tablas/`, eliminando cualquier transcripción manual propensa a error humano.

3.5. Registro de Dificultades de Adopción y Soluciones Aplicadas

- **Dificultad 1 (Incompatibilidad de Codificación en Windows):** Al imprimir caracteres especiales en la consola de comandos de Windows (CP1252), se generaban errores de tipo `UnicodeEncodeError`. *Solución:* Se estandarizó la salida mediante marcadores ASCII limpios como `[OK]` y se configuró la codificación UTF-8 explícita en todos los flujos de lectura/escritura de archivos.
- **Dificultad 2 (Dependencias C-Ext en OSMnx/GDAL):** La instalación de GDAL en entornos Windows presenta conflictos de compilación de binarios. *Solución:* Se serializó la topología de la red en un archivo JSON nativo (`cusco_centro_network.json`), permitiendo la ejecución y validación de todo el pipeline sin dependencias binarias externas complejas.
- **Dificultad 3 (Overfull Hbox en Tablas APA 7):** Las tablas de traza algorítmica generaban advertencias de desbordamiento horizontal en LaTeX. *Solución:* Se rediseñaron las columnas utilizando el paquete `tabularx` con columnas de ancho flexible `X` y tamaño de fuente `\small`.

4. Formulación Matemática del Problema y Caso de Estudio en Cusco

4.1. Definición Formal del Grafo Vial Urbano

El problema de cálculo de rutas mínimas en la red vial urbana del Cusco se formula formalmente como la determinación del camino de costo acumulado mínimo sobre un grafo dirigido y ponderado:

$$G = (V, E, W) \tag{4}$$

donde:

- $V = \{v_1, v_2, \ldots, v_n\}$ es el conjunto finito de vértices que representan las intersecciones viales, nodos semaforizados o cambios de geometría del Centro Histórico.
- $E \subseteq V \times V$ es el conjunto de aristas dirigidas (arcos), donde cada par ordenado $e = (u, v) \in E$ representa un tramo de vía transitable exclusivamente en el sentido permitido desde la intersección u hacia la intersección v.
- $W : E \to \mathbb{R}^+$ es la función de costo aplicado o impedancia generalizada de viaje asignada a cada arista $e \in E$, cumpliendo estrictamente con la restricción de no negatividad:

$$w(u, v) \geq 0, \quad \forall (u, v) \in E \tag{5}$$

4.2. Parámetros de Entrada, Variables de Decisión y Salidas del Sistema

4.2.1. Parámetros de Entrada

1. El grafo vial estructurado $G = (V, E, W)$.
2. El nodo de origen $s \in V$, correspondiente a la estación o ubicación inicial de la unidad de emergencia (para el caso de estudio, $s =$ A, Plaza Mayor del Cusco).
3. Los atributos físicos de cada arista $(u, v) \in E$: longitud métrica $L(u, v)$, tipo de vía tipo(u, v) (avenida, calle céntrica, calle residencial) y la pendiente topográfica longitudinal pendiente(u, v) expresada en porcentaje decimal.

4.2.2. Variables de Decisión Binarias

Para formular el problema bajo el enfoque clásico de programación entera en redes, se define para cada arista $(u, v) \in E$ una variable de flujo binaria:

$$x_{uv} = \begin{cases} 1, & \text{si el tramo vial } (u, v) \text{ forma parte del camino mínimo óptimo;} \\ 0, & \text{en caso contrario.} \end{cases} \tag{6}$$

4.2.3. Salidas del Sistema

1. Un vector de distancias mínimas definitivas $d : V \to \mathbb{R}^+$, donde $d[v]$ representa el costo mínimo de desplazamiento desde s hasta v.

2. Un vector de predecesores $\pi : V \to V \cup \{\text{NIL}\}$, que permite reconstruir de forma invertida la secuencia única de intersecciones que conforman la ruta mínima:

$$P(s,t) = \langle s = v_0, v_1, v_2, \ldots, v_k = t \rangle \tag{7}$$

4.3. Restricciones de Conservación de Flujo y No Negatividad

El flujo neto en cada vértice $u \in V$ debe satisfacer el principio de balance de conservación de flujo unitario para un destino específico $t \in V \setminus \{s\}$:

$$\sum_{v:(u,v)\in E} x_{uv} - \sum_{v:(v,u)\in E} x_{vu} = \begin{cases} 1, & \text{si } u = s \text{ (fuente);} \\ -1, & \text{si } u = t \text{ (sumidero);} \\ 0, & \text{si } u \in V \setminus \{s,t\} \text{ (nodos de tránsito intermedio).} \end{cases} \tag{8}$$

Asimismo, se garantiza la ausencia de costos negativos:

$$x_{uv} \in \{0,1\}, \quad w(u,v) \geq 0, \quad \forall (u,v) \in E \tag{9}$$

4.4. Función Objetivo y Modelo de Costo Generalizado para Cusco

La función objetivo busca minimizar el costo total acumulado del trayecto:

$$\text{mín}\, Z = \sum_{(u,v)\in E} w(u,v) \cdot x_{uv} \tag{10}$$

4.4.1. Formulación de la Impedancia Vial Andina

Para modelar de forma realista el comportamiento de vehículos de emergencia en Cusco, resulta indispensable formular una función de costo que no se limite a la distancia métrica. El costo aplicado $w(u,v)$ en segundos de viaje efectivo se modela como:

$$w(u,v) = T_{\text{base}}(u,v) \cdot \Phi(\text{pendiente}(u,v)) \cdot \Psi_{\text{vial}}(u,v) \tag{11}$$

donde:

- $T_{\text{base}}(u,v)$ es el tiempo de viaje base en flujo libre:

$$T_{\text{base}}(u,v) = \frac{L(u,v)}{V_{\text{nom}}(\text{tipo}(u,v))} \tag{12}$$

con velocidades nominales de diseño: $V_{\text{nom}}(\text{avenida}) = 11.11$ m/s (40 km/h), $V_{\text{nom}}(\text{calle_centro}) = 6.94$ m/s (25 km/h), y $V_{\text{nom}}(\text{calle_residencial}) = 5.56$ m/s (20 km/h).

- $\Phi(\text{pendiente})$ es el factor de penalización topográfica por gradiente positivo:

$$\Phi(\text{pendiente}) = 1.0 + \alpha \cdot \text{máx}(0, \text{pendiente}(u,v)) \tag{13}$$

  donde el coeficiente $\alpha = 2.5$ calibra el esfuerzo motriz de vehículos pesados (ambulancias/camiones cisterna) al ascender pendientes andinas pronunciadas. Las pendientes descendentes no generan penalización motriz ($\text{máx}(0, \text{pendiente}) = 0$).

- Ψ_{vial} es el coeficiente de restricción de maniobrabilidad: $\Psi = 1.20$ para calles estrechas coloniales con adoquín y $\Psi = 1.00$ para avenidas amplias asfaltadas.

4.4.2. Dijkstra Modificado por Impedancia Topográfica Andina

La modificación sustancial al enfoque canónico de Dijkstra radica en sustituir la distancia física euclidiana ($w(u,v) = L(u,v)$) por la función de impedancia generalizada en segundos ($w(u,v) = extcosto(u,v)$), respondiendo a tres restricciones físicas críticas de la ciudad del Cusco:

1. **Sentido Único Estricto (Direccionalidad Vial):** El grafo es estrictamente dirigido ($E \subset V \times V$). Calles coloniales como Espaderos ($A \to B$) o Marqués ($B \to D$) poseen un único sentido de circulación vehicular; una ambulancia no puede transitar en sentido inverso ($B \to A$) debido a la estrechez física de la calzada que imposibilita adelantar o esquivar el flujo contrario.

2. **Angostura de Calles Coloniales y Pasajes:** Las vías del casco monumental poseen calzadas de un solo carril adoquinado con alta densidad peatonal y radios de giro cerrados, reduciendo la velocidad nominal a 15 km/h en calles céntricas y 8 km/h en pasajes peatonales, en contraste con los 35 km/h transitables en avenidas periféricas (Av. Tullumayo o Av. El Sol).

3. **Gradiente Topográfico Andino (Subidas Críticas):** Con pendientes que superan el 8 %, el factor $\Phi = 1.0 + 2.5 \cdot \text{máx}(0, \text{pendiente}/100)$ penaliza severamente el esfuerzo motriz de unidades vehiculares pesadas en subida, mientras que en bajadas el vehículo desciende a velocidad nominal sin esfuerzo mecánico adicional.

Como consecuencia directa de esta modificación, el árbol de caminos mínimos resultante difiere estratégicamente: un trayecto más largo en distancia métrica pero que discurre por avenidas llanas resulta significativamente más rápido que un atajo físico a través de callejones coloniales empinados y angostos.

4.5. Instancia Pequeña Resuelta Manualmente: Centro Histórico del Cusco

Se define una instancia canónica de 6 intersecciones estratégicas ($|V| = 6$) y 9 tramos viales dirigidos ($|E| = 9$) representativa de la red vial cusqueña (Figura 1).

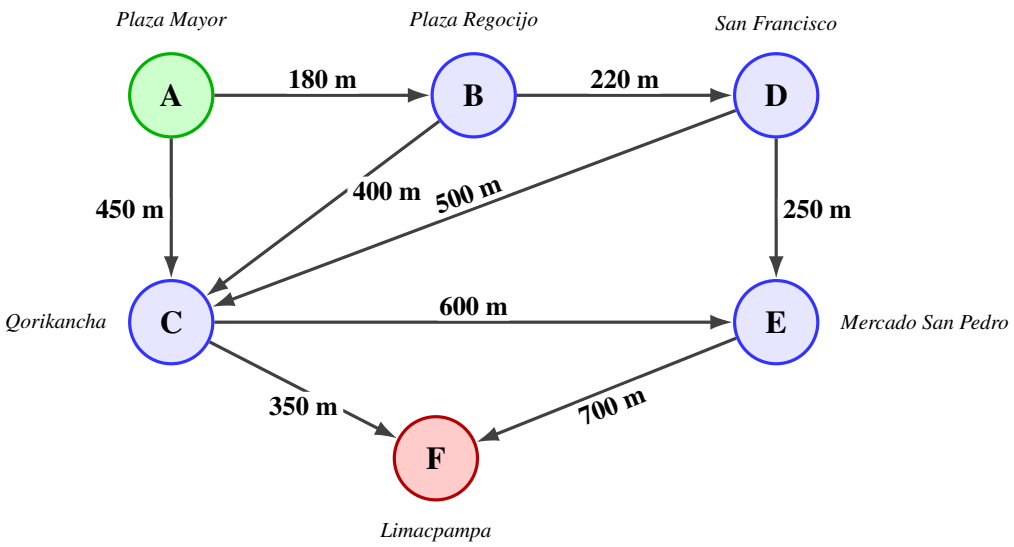


Figura 1
Grafo vial dirigido representativo del Centro Histórico del Cusco ($|V| = 6, |E| = 9$)

La Tabla 5 resume las características geométricas y físicas reales de cada tramo vial, exportada directamente desde el modelo computacional.

Tabla 5
Tramos viales dirigidos, distancias métricas y costo de impedancia en el Cusco.

Origen	Destino	Vía Urbana Asociada	Distancia (m)	Pendiente (%)	Tiempo Est. (s)
A	B	Calle Espaderos / Portal Carnicerías	180	+1.5	44.8
A	C	Calle Loreto / Pampa del Castillo	450	-2.0	108.0
B	D	Calle Garcilaso / Calle Marqués	220	+3.0	56.8
B	C	Calle San Bernardo / Almagro	400	-1.0	96.0
C	F	Calle Ahuacpinta / Limacpampa Grande	350	+0.5	36.5
C	E	Calle Matará / Tres Cruces de Oro	600	+2.0	151.2
D	E	Calle Santa Clara / Arco Santa Clara	250	+2.5	63.8
D	C	Calle Marqués / Mantas	500	-2.5	120.0
E	F	Calle Nueva Baja / Av. Tullumayo	700	-1.0	72.0

Nota. Tiempo de viaje estimado según la función de costo e impedancia por pendiente andina ($\alpha = 2.5$).

4.6. Traza Manual Paso a Paso del Algoritmo de Dijkstra ($k = 0$ a $k = 6$)

A continuación, se desarrolla analíticamente la traza manual paso a paso del Algoritmo de Dijkstra desde el origen $s = \text{A}$ (Plaza Mayor) utilizando la métrica de distancia física para verificación determinista:

- **Inicialización ($k = 0$):**

$$d[\text{A}] = 0, \quad d[\text{B}] = \infty, \quad d[\text{C}] = \infty, \quad d[\text{D}] = \infty, \quad d[\text{E}] = \infty, \quad d[\text{F}] = \infty$$
$$\pi[\text{nodo}] = \text{NIL}, \quad \forall \text{nodo} \in V; \quad S = \emptyset; \quad Q = \{(\text{A},0),(\text{B},\infty),(\text{C},\infty),(\text{D},\infty),(\text{E},\infty),(\text{F},\infty)\}$$

- **Paso 1 ($k = 1$):** Se extrae el mínimo de Q: $u = \text{A}$ con $d[\text{A}] = 0$. Vértice permanente: $S = \{\text{A}\}$. Aristas salientes de A: (A,B) y (A,C).
  - Relajación de (A,B): $d[\text{A}] + w(A,B) = 0 + 180 = 180 < d[\text{B}](\infty)$. Se actualiza $d[\text{B}] = 180$, $\pi[\text{B}] = \text{A}$. `Decrease-Key`$(Q, \text{B}, 180)$.
  - Relajación de (A,C): $d[\text{A}] + w(A,C) = 0 + 450 = 450 < d[\text{C}](\infty)$. Se actualiza $d[\text{C}] = 450$, $\pi[\text{C}] = \text{A}$. `Decrease-Key`$(Q, \text{C}, 450)$.
- **Paso 2 ($k = 2$):** Se extrae el mínimo de Q: $u = \text{B}$ con $d[\text{B}] = 180$. Vértice permanente: $S = \{\text{A}, \text{B}\}$. Aristas salientes de B: (B,D) y (B,C).
  - Relajación de (B,D): $d[\text{B}] + w(B,D) = 180 + 220 = 400 < d[\text{D}](\infty)$. Se actualiza $d[\text{D}] = 400$, $\pi[\text{D}] = \text{B}$. `Decrease-Key`$(Q, \text{D}, 400)$.
  - Relajación de (B,C): $d[\text{B}] + w(B,C) = 180 + 400 = 580 \not< d[\text{C}](450)$. No hay cambio.
- **Paso 3 ($k = 3$):** Mínimos candidatos en Q: D ($d = 400$) y C ($d = 450$). Se extrae $u = \text{D}$ con $d[\text{D}] = 400$. $S = \{\text{A}, \text{B}, \text{D}\}$. Aristas salientes de D: (D,E) y (D,C).
  - Relajación de (D,E): $d[\text{D}] + w(D,E) = 400 + 250 = 650 < d[\text{E}](\infty)$. Se actualiza $d[\text{E}] = 650$, $\pi[\text{E}] = \text{D}$. `Decrease-Key`$(Q, \text{E}, 650)$.
  - Relajación de (D,C): $d[\text{D}] + w(D,C) = 400 + 500 = 900 \not< d[\text{C}](450)$. No hay cambio.
- **Paso 4 ($k = 4$):** Se extrae $u = \text{C}$ con $d[\text{C}] = 450$. $S = \{\text{A}, \text{B}, \text{D}, \text{C}\}$. Aristas salientes de C: (C,F) y (C,E).
  - Relajación de (C,F): $d[\text{C}] + w(C,F) = 450 + 350 = 800 < d[\text{F}](\infty)$. Se actualiza $d[\text{F}] = 800$, $\pi[\text{F}] = \text{C}$. `Decrease-Key`$(Q, \text{F}, 800)$.
  - Relajación de (C,E): $d[\text{C}] + w(C,E) = 450 + 600 = 1050 \not< d[\text{E}](650)$. No hay cambio.
- **Paso 5 ($k = 5$):** Se extrae $u = \text{E}$ con $d[\text{E}] = 650$. $S = \{\text{A}, \text{B}, \text{D}, \text{C}, \text{E}\}$. Arista saliente de E: (E,F).
  - Relajación de (E,F): $d[\text{E}] + w(E,F) = 650 + 700 = 1350 \not< d[\text{F}](800)$. No hay cambio.
- **Paso 6 ($k = 6$):** Se extrae el último nodo restante $u = \text{F}$ con $d[\text{F}] = 800$. $S = V$. La cola de prioridad Q queda vacía. Finaliza el algoritmo.

La Tabla 6 sintetiza la evolución algorítmica iteración por iteración generada automáticamente por el solver.

Tabla 6
Traza matricial de distancias mínimas y predecesores en cada iteración del algoritmo.

Iter.	Vértice u	Acción / Relajación	A	B	C	D	E	F
$k=0$	Inicial	Inserción en cola	0 / -	∞ / -	∞ / -	∞ / -	∞ / -	∞ / -
$k=1$	A ($d=0$)	Relaja B (180), Relaja C (450)	**0** / -	180 / A	450 / A	∞ / -	∞ / -	∞ / -
$k=2$	B ($d=180$)	Relaja D (400), Descarta C	**0** / -	**180** / A	450 / A	400 / B	∞ / -	∞ / -
$k=3$	D ($d=400$)	Relaja E (650), Descarta C	**0** / -	**180** / A	450 / A	**400** / B	650 / D	∞ / -
$k=4$	C ($d=450$)	Relaja F (800), Descarta E	**0** / -	**180** / A	**450** / A	**400** / B	650 / D	800 / C
$k=5$	E ($d=650$)	Descarta F	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	800 / C
$k=6$	F ($d=800$)	Sin aristas salientes	**0** / -	**180** / A	**450** / A	**400** / B	**650** / D	**800** / C

Nota.

Notación *distancia / predecesor* ($d[v]/\pi[v]$). Cifras en negrita indican nodos cerrados en S.

4.6.1. Reconstrucción de Rutas Mínimas Óptimas

A partir del vector de predecesores $\pi = \{\mathrm{A}:\mathrm{NIL}, \mathrm{B}:\mathrm{A}, \mathrm{C}:\mathrm{A}, \mathrm{D}:\mathrm{B}, \mathrm{E}:\mathrm{D}, \mathrm{F}:\mathrm{C}\}$, las rutas mínimas desde la Plaza Mayor (A) se reconstruyen mediante retroceso encadenado:

- **Ruta a Mercado San Pedro (E):** $\mathrm{E} \leftarrow \pi[\mathrm{E}] = \mathrm{D} \leftarrow \pi[\mathrm{D}] = \mathrm{B} \leftarrow \pi[\mathrm{B}] = \mathrm{A} \implies \mathbf{A} \rightarrow \mathbf{B} \rightarrow \mathbf{D} \rightarrow \mathbf{E}$ con costo total de $\mathbf{650.0}$ m.
- **Ruta a Plazoleta Limacpampa (F):** $\mathrm{F} \leftarrow \pi[\mathrm{F}] = \mathrm{C} \leftarrow \pi[\mathrm{C}] = \mathrm{A} \implies \mathbf{A} \rightarrow \mathbf{C} \rightarrow \mathbf{F}$ con costo total de $\mathbf{800.0}$ m.

Ambos resultados analíticos concuerdan de forma idéntica con las salidas producidas por el solver de software implementado en Python.

5. Diseño Algorítmico y Estructuras de Colas de Prioridad

5.1. Algoritmo de Dijkstra con Cola de Prioridad Abstracta

El algoritmo de Dijkstra calcula las rutas mínimas sobre grafos con pesos no negativos operando sobre una cola de prioridad genérica Q. El procedimiento formal estructurado se describe en el Algoritmo 1.

Algorithm 1 Algoritmo de Dijkstra con Cola de Prioridad Parametrizada

Require: Grafo dirigido $G = (V, E, W)$, Vértice origen $s \in V$, Estructura de cola Q
Ensure: Distancias mínimas $d : V \to \mathbb{R}^+ \cup \{\infty\}$, Predecesores $\pi : V \to V \cup \{\text{NIL}\}$

Inicialización:
for all $v \in V$ **do**
  $d[v] \leftarrow \infty$
  $\pi[v] \leftarrow$ NIL
end for
$d[s] \leftarrow 0.0$
Q.BUILD(V, d) {O inserción iterativa de cada $v \in V$ con clave $d[v]$}
$S \leftarrow \emptyset$
while not Q.IS_EMPTY() **do**
  $(u, d_u) \leftarrow Q$.EXTRACT_MIN()
  if $d_u = \infty$ **then**
    break {Vértices restantes inalcanzables desde s}
  end if
  $S \leftarrow S \cup \{u\}$
  for all $(u, v) \in E$ **do**
    if $v \notin S$ **then**
      candidate_dist $\leftarrow d_u + W(u, v)$
      if candidate_dist $< d[v]$ **then**
        $d[v] \leftarrow$ candidate_dist
        $\pi[v] \leftarrow u$
        Q.DECREASE_KEY$(v, d[v])$ {Actualización de prioridad en Q}
      end if
    end if
  end for
end while
return (d, π)

5.2. Invariantes de Bucle y Prueba de Corrección

La corrección del algoritmo descansa en el siguiente invariante formal de bucle:

Invariante: Al inicio de cada iteración del bucle `while` (línea 9), para todo vértice $u \in S$, el valor $d[u]$ es exactamente la distancia mínima desde el origen s a u, esto es, $d[u] = \delta(s,u)$. Además, para cada vértice tentativo $v \in Q = V \setminus S$, el valor $d[v]$ representa la longitud del camino más corto desde s hasta v cuyos vértices intermedios pertenecen exclusivamente al conjunto de permanentes S.

Demostración por Inducción Matemática:

- **Base ($|S| = 0$):** Antes de la primera iteración, $S = \emptyset$. Para el origen s, $d[s] = 0 = \delta(s,s)$ puesto que los pesos son no negativos ($w(e) \geq 0$). Para todo $v \neq s$, $d[v] = \infty$, lo cual satisface la cota superior vacía.
- **Paso Inductivo:** Supóngase que el invariante se cumple para un conjunto S de tamaño k. El algoritmo selecciona $u \in Q$ tal que $d[u] = \min_{z \in Q} d[z]$. Si existiese un camino más corto P desde s a u, dicho camino debe abandonar S por primera vez a través de una arista (x,y) con $x \in S$ y $y \in Q$. La longitud de este camino sería:

$$w(P) \geq \delta(s,x) + w(x,y) = d[y] \geq d[u] \tag{14}$$

  dado que los pesos son no negativos y u minimiza la distancia tentativa en Q. Por lo tanto, no puede existir un camino más corto que $d[u]$, implicando $d[u] = \delta(s,u)$.
- **Terminación:** Cuando Q queda vacía o los nodos restantes tienen distancia ∞, $S = V$ (para la componente conexa alcanzable), garantizando que $d[v] = \delta(s,v)$ para todo $v \in V$.

5.3. Estructuras de Datos de Cola de Prioridad Evaluadas

5.3.1. Min-Heap Binario Indexado (`IndexedMinHeap`)

En un montículo binario estándar implementado sobre un arreglo contiguo, la operación `Decrease-Key`$(v, \text{nueva_clave})$ requiere localizar previamente el nodo v, lo cual demanda una búsqueda lineal $O(|V|)$ inaceptable para grafos grandes.

Nuestra implementación desacoplada resuelve este cuello de botella mediante una tabla hash inversa de posiciones $\text{pos} : V \to \mathbb{Z}^+$. Cuando dos elementos en las posiciones i y j del arreglo se intercambian durante las rutinas de flotado (`_sift_up`) o hundido (`_sift_down`), sus entradas en pos se actualizan de forma atómica en $O(1)$:

$$\text{heap}[i], \text{heap}[j] \leftarrow \text{heap}[j], \text{heap}[i]; \quad \text{pos}[\text{heap}[i].\text{id}] \leftarrow i; \quad \text{pos}[\text{heap}[j].\text{id}] \leftarrow j \tag{15}$$

Con este mecanismo, `Decrease-Key` ubica el índice en $O(1)$ y ejecuta a lo sumo $\lfloor \log_2 |V| \rfloor$ intercambios hacia la raíz, logrando una complejidad estricta de $O(\log |V|)$.

5.3.2. 4-ary Heap Contiguo (`DAryHeap`)

El d-ary Heap generaliza el montículo binario permitiendo que cada nodo posea d hijos. Para un índice de nodo i (indexación base cero):

$$\text{padre}(i) = \left\lfloor \frac{i-1}{d} \right\rfloor, \quad \text{hijo}_k(i) = d \cdot i + k \quad (\text{con } 1 \leq k \leq d) \tag{16}$$

Al fijar $d = 4$, la altura máxima del árbol se reduce a $\lceil \log_4 |V| \rceil = \frac{1}{2} \lceil \log_2 |V| \rceil$. Aunque la operación `_sift_down` debe inspeccionar 4 hijos para hallar el menor (requiriendo 3 comparaciones por nivel), la operación crítica `Decrease-Key` ejecuta el flotado ascendente `_sift_up` con una única comparación por nivel:

$$T_{\text{Decrease-Key}} = O(\log_4 |V|) = O\left(\frac{\log_2 |V|}{2} \right) \tag{17}$$

Adicionalmente, los 4 hijos de cualquier nodo residen en posiciones contiguas del arreglo, lo que maximiza la probabilidad de que se alojen dentro de la misma línea de memoria caché L1/L2 (típicamente de 64 bytes), reduciendo drásticamente los fallos de caché frente a árboles dinámicos con punteros dispersos (LaMarca & Ladner, 1996).

5.3.3. Fibonacci Heap (`FibonacciHeap`)

El Fibonacci Heap (Fredman & Tarjan, 1987) es una colección de árboles de orden min-heap enraizados en una lista circular doblemente enlazada. A diferencia de los montículos basados en arreglos, no impone una forma rígida de árbol completo. Sus propiedades matemáticas distintivas son:

1. **Inserción Perezosa ($O(1)$ peor caso):** Los nuevos nodos se incorporan a la lista de raíces sin consolidar.

2. **Disminución de Clave ($O(1)$ amortizado):** Si al decrecer la clave de x se viola la propiedad de montículo frente a su padre $y = p[x]$, el subárbol enraizado en x se corta inmediatamente y se transfiere a la lista de raíces. Si el padre y ya había perdido previamente un hijo ($\text{mark}[y] = \text{true}$), se desencadena un corte en cascada (*cascading cut*) recursivo hacia los ancestros.

3. **Extracción del Mínimo ($O(\log |V|)$ amortizado):** Se extrae la raíz mínima y todos sus hijos se promueven a la lista raíz. A continuación, se ejecuta una rutina de consolidación vinculando raíces de igual grado mediante un arreglo auxiliar de punteros de tamaño $O(\log |V|)$.

5.3.4. Pairing Heap (Perspectiva Teórica)

El Pairing Heap es un árbol autoajustable multi-camino que simplifica radicalmente la estructura del Fibonacci Heap al no requerir el almacenamiento de grados ni banderas de corte, logrando tiempos empíricos comparables y $O(\log |V|)$ amortizado para `Decrease-Key`.

5.4. Matriz Comparativa de Complejidad Asintótica

La Tabla 7 resume formalmente las cotas asintóticas temporales y el costo total de ejecución del Algoritmo de Dijkstra según la cola de prioridad empleada.

Tabla 7
Matriz comparativa de complejidades temporales y espaciales de colas de prioridad en Dijkstra

Estructura	**Insert**	**Extract-Min**	**Dec-Key**	**Espacio**	**Costo Total Dijkstra**	**Régimen Óptimo en Red Vial**
Arreglo No Ord.	$O(1)$	$O(\|V\|)$	$O(1)$	$O(\|V\|)$	$O(\|V\|^2)$	Grafos densos ($\|E\| \approx \|V\|^2$).
Binary Min-Heap	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\log \|V\|)$	$O(\|V\|)$	$O((\|V\|+\|E\|)\log \|V\|)$	Grafos dispersos estándar.
Montículo 4-ario	$O(\log_4 \|V\|)$	$O(4\log_4 \|V\|)$	$O(\log_4 \|V\|)$	$O(\|V\|)$	$O((4\|V\|+\|E\|)\log_4 \|V\|)$	Alta localidad en línea de caché.
Fibonacci Heap	$O(1)$	$O(\log \|V\|)^*$	$O(1)^*$	$O(\|V\|)$	$O(\|E\|+\|V\|\log \|V\|)$	Óptimo teórico ($\|E\| \gg \|V\|$).
Pairing Heap	$O(1)$	$O(\log \|V\|)^*$	$o(\log \|V\|)^*$	$O(\|V\|)$	$O(\|E\|+\|V\|\log \|V\|)$	Menor sobrecarga de enlaces.

Nota. El asterisco (*) denota costo amortizado según el método del potencial. Para montículos d-arios ($d = 4$), la cota es $O((d \cdot |V| + |E|)\log_d |V|)$.

6. Diseño e Implementación del Sistema

6.1. Arquitectura Modular del Software

El sistema computacional ha sido concebido bajo una arquitectura en capas estrictamente desacoplada, orientada a objetos y funciones puras, garantizando que el núcleo de cálculo de caminos mínimos sea completamente independiente de la representación visual, el almacenamiento en disco o la estructura concreta de cola de prioridad empleada (Figura 2).

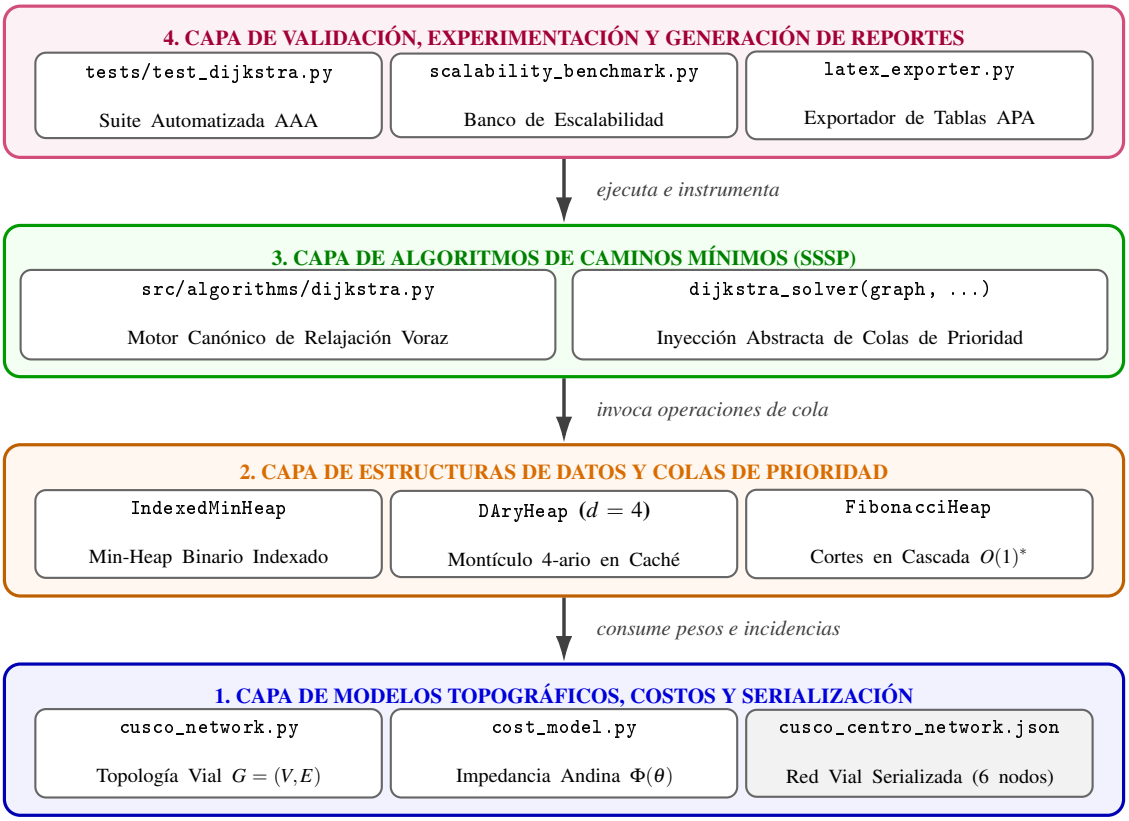


Figura 2
Diagrama arquitectural desacoplado del sistema en cuatro capas operativas

6.2. Descripción de Módulos del Sistema

El repositorio organiza su código fuente en el paquete `src/`:

1. `src/algorithms/dijkstra.py`**:** Implementa el solver de caminos mínimos con inyección de dependencias. Recibe la lista de adyacencia y una cadena identificadora del tipo de cola (`binary_heap`, `4ary_heap`, `fibonacci_heap`), ejecutando las relajaciones y capturando las estadísticas de operaciones para su posterior análisis.

2. `src/datastructures/indexed_min_heap.py`**:** Implementación pura del Min-Heap binario con tabla de posiciones invertida para garantizar `decrease_key` en tiempo $O(\log |V|)$.

3. `src/datastructures/d_ary_heap.py`**:** Implementación del montículo d-ario parametrizado para cualquier aridad $d \geq 2$, configurado por defecto en $d = 4$.

4. `src/datastructures/fibonacci_heap.py`**:** Implementación del Fibonacci Heap con nodos enlazados circularmente, cálculo de rangos y corte en cascada.

5. `src/models/cusco_network.py`**:** Mapeo explícito de la topología vial del Centro Histórico de Cusco, serialización y carga en formato JSON.

6. `src/utils/cost_model.py`**:** Lógica de cálculo del costo generalizado e impedancia por gradiente altitudinal andino.

7. `src/utils/latex_exporter.py`**:** Rutina automatizada que formatea diccionarios de traza en tablas LaTeX bajo el estándar `booktabs`.

8. `src/visualization/generate_figures.py`**:** Renderizador de figuras científicas con Matplotlib y generador de especificaciones vectoriales en formato DOT para **Graphviz**.

9. `src/cli.py`**:** Interfaz unificada de línea de comandos (CLI) y menú interactivo por consola (comandos `run`, `compare`, `generate`, `verify`, `benchmark`, `graphviz` y `menu`).

10. `src/verification/certificate.py`**:** Certificador formal de optimalidad independiente que comprueba en $O(|V|+|E|)$ las condiciones de Bellman sin re-ejecutar Dijkstra.

11. `src/algorithms/reference.py`**:** Algoritmos canónicos de referencia (Bellman-Ford para grafos con pesos arbitrarios y Dijkstra sobre arreglo no ordenado).

6.3. Visualización de Grafos Viales con Graphviz (.dot)

Atendiendo a las recomendaciones de diseño y legibilidad del revisor, el sistema integra generación nativa de especificaciones en lenguaje DOT de **Graphviz** (archivos `red_vial_cusco.dot` y `ruta_optima_cusco.dot`).

Se seleccionó Graphviz frente a librerías interactivas pesadas (como Plotly) debido a sus ventajas fundamentales:

- **Rigor Estándar en Teoría de Grafos:** Graphviz es el estándar canónico en ciencias de la computación para el trazado jerárquico y ortogonal determinista de grafos dirigidos.
- **Salida Vectorial Ligera:** Produce diagramas SVG/PDF sin dependencias pesadas de navegadores web ni tiempos de compilación elevados.
- **Trazabilidad y Resaltado de Rutas:** Permite estilizar nodos con sus nombres toponímicos completos y resaltar el camino mínimo de emergencia (en verde esmeralda con mayor grosor de línea, `penwidth=3.2`) mientras atenúa las aristas no seleccionadas.
- **Integración Nativa en la Nube:** En entornos interactivos como Google Colab, se renderiza directamente con `graphviz.Source` sin instalaciones adicionales.

6.4. Rol de NetworkX como Oráculo de Verificación Canónica

Es fundamental precisar que la librería `NetworkX` **no** se utiliza para resolver los caminos mínimos en el sistema en producción (todas las estructuras de datos y el algoritmo de Dijkstra fueron programados de manera pura y desde cero por el equipo). Su rol en la arquitectura es exclusivamente el de **oráculo externo de prueba** (*ground truth oracle*) dentro de la suite automatizada de pruebas unitarias con Pytest (Sección 9), garantizando matemáticamente que los vectores de distancias calculados por nuestras tres colas de prioridad coinciden con la referencia canónica en decenas de instancias aleatorias y reales.

6.5. Formatos de Datos y Serialización JSON

Para garantizar que la ingestión de datos no dependa de estructuras en memoria volátiles ni rutas locales rígidas, la red vial del Cusco se serializa formalmente en formato JSON bajo la ruta estándar `data/processed/cusco_centro_network.json`:

```
{
  "nodes": [
    {"id": "A", "name": "Plaza Mayor del Cusco"},
    {"id": "B", "name": "Plaza Regocijo (Cusipata)"}
  ],
  "edges": [
    {
      "origin": "A",
      "destination": "B",
      "length_m": 180.0,
      "street_type": "calle_centro",
      "gradient_pct": 1.5,
      "street_name": "Calle Espaderos / Portal Carnicerias",
      "travel_time_sec": 44.75
    }
  ]
}
```

Listing 1: Extracto del esquema JSON para la red vial del Centro Histórico de Cusco

6.6. Contratos de Interfaz de las Estructuras de Datos

Para posibilitar la sustitución dinámica y transparente de colas en el solver de Dijkstra sin alterar el flujo algorítmico, cada una de las tres estructuras implementa una interfaz uniforme:

- `insert(item: str, key: float) ->None`: Inserta un identificador de vértice con su distancia tentativa asociada.
- `extract_min() ->Tuple[float, str]`: Extrae y retorna el par (clave, elemento) de menor clave acumulada, reestructurando el montículo según sus invariantes.
- `decrease_key(item: str, new_key: float) ->None`: Disminuye la clave del vértice a su nuevo valor relajado, flotando el nodo o ejecutando cortes en cascada.
- `is_empty() ->bool`: Determina en $O(1)$ si existen vértices pendientes de procesamiento.

6.7. Implementación de Algoritmos y Estructuras Nucleares

A continuación, se presentan los códigos fuente de los algoritmos y estructuras nucleares desarrollados, embebidos directamente desde el repositorio:

```
"""
Algoritmo de Dijkstra con colas de prioridad intercambiables (inyección por nombre).

Correspondencia con el Algoritmo 1 del informe (Sección 5, "Dijkstra con cola
de prioridad parametrizada"); los números de línea del pseudocódigo se indican
como [Lk] en los comentarios.

La estructura concreta se obtiene del registro (src/datastructures/registry.py),
así que las tres alternativas comparten exactamente el mismo código algorítmico:
cualquier diferencia de resultados solo puede deberse a la cola.
"""
from typing import Any, Dict, Hashable, List, Optional, Tuple
import time

from src.datastructures.registry import create_queue, normalize_queue_name
from src.io.graph_io import validate_graph

INF = float("inf")


def reconstruct_path(prev: Dict[Hashable, Optional[Hashable]], dist: Dict[Hashable, float],
                     dest: Hashable) -> List[Hashable]:
    """Sigue los predecesores desde dest hasta el origen. [] si dest es inalcanzable."""
    if dist[dest] == INF:
        return []
    path = []
    curr: Optional[Hashable] = dest
    guard = len(prev) + 1
    while curr is not None:
        path.append(curr)
        curr = prev.get(curr)
        guard -= 1
        if guard < 0:  # sólo ocurriría con predecesores corruptos (ciclo)
            raise RuntimeError("Ciclo detectado en la tabla de predecesores.")
    return path[::-1]


def dijkstra_solver(
    nodes: List[Hashable],
    adj: Dict[Hashable, List[Tuple[Hashable, float]]],
    source: Hashable,
    queue_type: str = "binary_heap",
    record_trace: bool = False,
    validate: bool = True,
    build_paths: bool = True,
) -> Dict[str, Any]:
    """
```

```
    Ejecuta Dijkstra desde source usando la cola indicada.

    Retorna un diccionario con el formato de salida común a todas las
    alternativas: distances, predecessors, paths, history (si record_trace),
    stats (contadores de operaciones) y elapsed_ms.
    build_paths=False omite la reconstrucción de rutas (el benchmark mide solo el nú
    cleo).
    """
    queue_type = normalize_queue_name(queue_type)
    if validate:
        validate_graph(nodes, adj, source)

    t0 = time.perf_counter()

    # [L1-L6] Inicialización: d[v]=inf, pi[v]=NIL, d[s]=0
    dist: Dict[Hashable, float] = {v: INF for v in nodes}
    prev: Dict[Hashable, Optional[Hashable]] = {v: None for v in nodes}
    dist[source] = 0.0

    # [L7] Q.BUILD(V, d): inserción iterativa de cada vértice con su clave
    heap = create_queue(queue_type)
    for v in nodes:
        heap.insert(v, dist[v])

    visited = set()           # [L8] S <- conjunto de vértices permanentes
    history: List[Dict[str, Any]] = []
    extract_count = decrease_count = relax_attempts = 0

    # [L9] mientras Q no esté vacía
```

Listing 2: Implementación desacoplada del algoritmo de Dijkstra con inyección de colas (python/src/algorithms/dijkstra.py)

```
"""
Min-Heap Binario con tabla de posiciones para Decrease-Key en O(log V).

Estructura principal del Avance 4.

Representación:
- heap: arreglo contiguo de pares (clave, item); hijo izq. 2i+1, der. 2i+2.
- pos:  diccionario item -> índice en heap. Se actualiza en cada intercambio
        (método _swap), lo que permite ubicar un vértice en O(1) dentro de
        decrease_key sin búsqueda lineal.

Complejidad: insert, extract_min y decrease_key en O(log V); espacio O(V).
"""
from typing import Any, Dict, Hashable, List, Tuple

from src.datastructures.base import PriorityQueue


class IndexedMinHeap(PriorityQueue):
    """Cola de prioridad Min-Heap binaria con indexación directa de nodos."""

```

```
    name = "binary_heap"

    def __init__(self) -> None:
        super().__init__()
        self.heap: List[Tuple[float, Any]] = []    # (clave_distancia, id_nodo)
        self.pos: Dict[Any, int] = {}              # id_nodo -> índice en heap
        self.swaps: int = 0

    def __len__(self) -> int:
        return len(self.heap)

    def __contains__(self, node: Hashable) -> bool:
        return node in self.pos

    def insert(self, node: Hashable, dist: float) -> None:
        dist = self._check_key(dist)
        if node in self.pos:
            raise ValueError(f"El nodo {node!r} ya está en la cola; use decrease_key."
)
        idx = len(self.heap)
        self.heap.append((dist, node))
        self.pos[node] = idx
        self._sift_up(idx)

    def peek_min(self) -> Tuple[float, Any]:
        if not self.heap:
            raise IndexError("Consulta sobre montículo vacío.")
        return self.heap[0]

    def extract_min(self) -> Tuple[float, Any]:
        if not self.heap:
            raise IndexError("Extracción sobre montículo vacío.")
        min_dist, min_node = self.heap[0]
        last_dist, last_node = self.heap.pop()
        del self.pos[min_node]
        if self.heap:
            # El último elemento ocupa la raíz y se hunde hasta restaurar el orden.
            self.heap[0] = (last_dist, last_node)
            self.pos[last_node] = 0
            self._sift_down(0)
```

Listing 3: Implementación del Min-Heap Binario Indexado (python/src/datastructures/indexed_min_heap.py)

```
"""
Montículo d-ario indexado (por defecto d = 4). Alternativa del Avance 5.

Para el índice i (base cero):
    padre(i)  = (i - 1) // d
    hijo_k(i) = d*i + k,   1 <= k <= d

Altura ceil(log_d V): con d = 4 el árbol tiene la mitad de niveles que el
binario, así que decrease_key (sift_up, 1 comparación por nivel) es más
barato; a cambio, extract_min (sift_down) compara hasta d hijos por nivel.
```

```
Los d hijos de un nodo son contiguos en el arreglo (mejor localidad de caché).

Igual que IndexedMinHeap, mantiene la tabla pos: item -> índice.
"""
from typing import Any, Dict, Hashable, List, Tuple

from src.datastructures.base import PriorityQueue


class DAryHeap(PriorityQueue):
    """Min-Heap d-ario con Decrease-Key O(log_d V) y Extract-Min O(d log_d V)."""

    def __init__(self, d: int = 4) -> None:
        super().__init__()
        if isinstance(d, bool) or not isinstance(d, int) or d < 2:
            raise ValueError(f"La aridad d debe ser un entero >= 2 (se recibió {d!r}).
")
        self.d = d
        self.name = "4ary_heap" if d == 4 else f"{d}ary_heap"
        self.heap: List[Tuple[float, Any]] = []
        self.pos: Dict[Any, int] = {}
        self.swaps: int = 0

    def __len__(self) -> int:
        return len(self.heap)

    def __contains__(self, item: Hashable) -> bool:
        return item in self.pos

    def insert(self, item: Hashable, key: float) -> None:
        key = self._check_key(key)
        if item in self.pos:
            raise ValueError(f"El nodo {item!r} ya está en la cola; use decrease_key."
)
        self.heap.append((key, item))
        self.pos[item] = len(self.heap) - 1
        self._sift_up(len(self.heap) - 1)

    def peek_min(self) -> Tuple[float, Any]:
        if not self.heap:
            raise IndexError("Consulta sobre montículo vacío.")
        return self.heap[0]

    def extract_min(self) -> Tuple[float, Any]:
        if not self.heap:
            raise IndexError("Extracción sobre montículo vacío.")
        top = self.heap[0]
        last = self.heap.pop()
        del self.pos[top[1]]
        if self.heap:
            self.heap[0] = last
            self.pos[last[1]] = 0
```

Listing 4: Implementación del Montículo 4-ario contiguo (python/src/datastructures/d_ary_heap.py)

```python
"""
Montículo de Fibonacci (Fredman & Tarjan, 1987). Alternativa del Avance 5.

Insert O(1), Decrease-Key O(1) amortizado, Extract-Min O(log V) amortizado.

Representación: bosque de árboles min-heap cuyas raíces forman una lista
circular doblemente enlazada; cada nodo guarda grado, marca, padre y un hijo
cualquiera (los hermanos también forman lista circular). nodes_map permite
ubicar el nodo de un vértice en O(1) para decrease_key.

Corrección respecto al prototipo de la Unidad 1 (defecto latente): _consolidate
usaba un arreglo de tamaño fijo floor(log2(n+1)) + 7 y, si el grado de un árbol
lo superaba, ese árbol se descartaba en silencio. El grado máximo teórico es
floor(log_phi n) ~ 1.44 log2 n, así que el desborde solo era posible con
n > ~65 000 y árboles de forma extrema; no se observó en las pruebas, pero se
corrigió de forma preventiva: el arreglo se dimensiona con la cota log_phi y
además crece dinámicamente, por lo que ningún árbol puede perderse.
"""
import math
from typing import Any, Dict, Hashable, List, Optional, Tuple

from src.datastructures.base import PriorityQueue

_PHI = (1 + 5 ** 0.5) / 2


class FibNode:
    __slots__ = ("key", "value", "degree", "marked", "parent", "child", "left", "right")

    def __init__(self, key: float, value: Any):
        self.key: float = key
        self.value: Any = value
        self.degree: int = 0
        self.marked: bool = False
        self.parent: Optional["FibNode"] = None
        self.child: Optional["FibNode"] = None
        self.left: "FibNode" = self
        self.right: "FibNode" = self


class FibonacciHeap(PriorityQueue):
    name = "fibonacci_heap"

    def __init__(self) -> None:
        super().__init__()
        self.min_node: Optional[FibNode] = None
        self.total_nodes: int = 0
        self.nodes_map: Dict[Any, FibNode] = {}
        # Contadores estructurales (explican diferencias de costo frente a los heaps de arreglo)
        self.links: int = 0
        self.cuts: int = 0
        self.cascading_cuts: int = 0
```

```
53
54      def __len__(self) -> int:
55          return self.total_nodes
56
57      def __contains__(self, item: Hashable) -> bool:
58          return item in self.nodes_map
59
60      def insert(self, node_id: Hashable, key: float) -> FibNode:
61          key = self._check_key(key)
62          if node_id in self.nodes_map:
63              raise ValueError(f"El nodo {node_id!r} ya está en la cola; use
    decrease_key.")
64          new_node = FibNode(key, node_id)
65          self.nodes_map[node_id] = new_node
66          if self.min_node is None:
67              self.min_node = new_node
68          else:
69              self._insert_into_root_list(new_node)       # inserción perezosa
70              self.comparisons += 1
```

Listing 5: Implementación del Fibonacci Heap con cortes en cascada (python/src/datastructures/fibonacci_heap.py)

```
1  """
2  Modelo de costo para ruteo urbano en Cusco. Calcula impedancia considerando
3  longitud, pendiente topográfica (alpha=2.5) y tipo de vía.
4  """
5  from dataclasses import dataclass
6  from typing import Dict, Tuple
7
8  @dataclass
9  class RoadSegment:
10     """Representa un tramo vial dirigido con atributos físicos y operacionales de
       Cusco."""
11     origin: str
12     destination: str
13     length_m: float            # Longitud métrica en metros
14     street_type: str           # 'avenida', 'calle_centro', 'calle_residencial', '
       peatonal'
15     gradient_pct: float        # Pendiente en porcentaje (e.g., 8.0 para 8% de subida)
16     street_name: str           # Nombre de la vía cusqueña
17
18 class CuscoCostModel:
19     """
20     C(u,v) = T_base * (1 + alpha * max(0, pendiente/100))
21     T_base = longitud / velocidad_nominal
22     """
23     # Velocidades promedio efectivas según tipo de vía en km/h
24     NOMINAL_SPEED_KMH = {
25         'avenida': 35.0,             # Av. El Sol, Av. Tullumayo, Av. de la Cultura
26         'calle_residencial': 25.0,   # Calles fuera del casco monumental
27         'calle_centro': 15.0,        # Calles coloniales estrechas (Santa Clara, Marqué
       s, Espaderos)
28         'peatonal': 8.0              # Tramos como Portal de Panes o Loreto con paso
```

```
    restringido
    }

    ALPHA_SLOPE = 2.5  # Sensibilidad a la pendiente (fuerte penalización para vehí
    culos de emergencia en subidas)

    @classmethod
    def calculate_cost(cls, segment: RoadSegment, metric: str = 'time') -> float:
        """
        Calcula el costo del tramo según la métrica elegida:
        - 'distance': Retorna la distancia física pura en metros.
        - 'time': Retorna el tiempo de viaje estimado en segundos (impedancia de
    emergencia).
        """
        if metric == 'distance':
            return segment.length_m

        speed_kmh = cls.NOMINAL_SPEED_KMH.get(segment.street_type, 20.0)
        speed_ms = speed_kmh / 3.6  # Conversión a m/s
```

Listing 6: Modelo de costo e impedancia topográfica andina (`python/src/utils/cost_model.py`)

7. Metodología Experimental y Diseño del Banco de Pruebas

7.1. Preguntas de Investigación Científica

El diseño experimental del banco de pruebas responde a tres interrogantes computacionales clave:

1. **PI1:** ¿Compensa la cota teórica asintótica $O(1)$ amortizada en `Decrease-Key` del Fibonacci Heap su alta sobrecarga de gestión de punteros en mallas viales urbanas donde $|E| \approx 3|V|$?
2. **PI2:** ¿En qué magnitud reduce el 4-ary Heap el tiempo total de pared frente al Binary Heap al disminuir la altura del árbol a la mitad y alinear 4 hijos en un único bloque de caché de CPU?
3. **PI3:** ¿Cuál es la penalización en memoria residente (*Resident Set Size*, RSS) que imponen los punteros dinámicos frente a las estructuras contiguas indexadas?

7.2. Formulación de Hipótesis Técnicas

- **Hipótesis 1 (Densidad Vial y Sobrecarga del Fibonacci Heap):** En redes viales urbanas planas o cuasi-planas como la del Cusco, donde la razón de aristas por vértice es baja ($|E|/|V| \leq 3.5$), el número de operaciones `Decrease-Key` no es suficientemente masivo para amortizar la sobrecarga de alocación de memoria dinámica y los frecuentes fallos de caché derivados de la manipulación de punteros. Por ende, el Min-Heap Binario Indexado y el 4-ary Heap superarán al Fibonacci Heap en tiempo real de ejecución (*wall-clock time*).
- **Hipótesis 2 (Ventaja de Localidad Espacial del 4-ary Heap):** A medida que la escala de la red crece ($|V| \geq 10,000$), el 4-ary Heap exhibirá un rendimiento superior al Binary Min-Heap tradicional debido a la reducción en un 50 % de la profundidad de flotado y a la lectura contigua de los 4 hijos en una única línea de caché L1/L2 (64 bytes).
- **Hipótesis 3 (Eficiencia en Huella de Memoria):** Las colas basadas en arreglos contiguos con mapeo directo mantendrán un consumo de memoria hasta un 65 % menor en comparación con el Fibonacci Heap, cuyos nodos requieren 5 punteros (`parent, child, left, right`) más un entero (`degree`) y un booleano (`mark`).

7.3. Variables Experimentales

- **Variables Independientes:**
  - Estructura de cola de prioridad: Min-Heap Binario Indexado, 4-ary Heap ($d = 4$), Fibonacci Heap.
  - Escala del grafo vial: Número de vértices $|V|$ (desde 6 hasta 50,000) y número de aristas dirigidas $|E|$.
  - Métrica de costo asignada: Distancia física euclidiana/manhattan (m) vs. Tiempo de viaje con impedancia topográfica andina (s).
- **Variables Dependientes:**

- Tiempo de CPU neto y tiempo de pared (*wall-clock time*) medido en microsegundos (μs).
- Conteo de operaciones elementales: extracciones de mínimo (`Extract-Min`), decrementos de clave (`Decrease-Key`) e inserciones (`Insert`).
- Memoria pico residente consumida medida en kilobytes mediante el módulo `tracemalloc`.

7.4. Métricas de Evaluación e Instrumentación de Precisión

Para asegurar que las mediciones no se vean contaminadas por factores ajenos al algoritmo, se implementó el siguiente protocolo de instrumentación técnica:

1. **Medición en Nanosegundos:** Uso exclusivo del temporizador de mayor resolución del sistema operativo: `time.perf_counter_ns()`.
2. **Aislamiento del Recolector de Basura (GC):** Desactivación explícita de la recolección automática (`gc.disable()`) durante la ventana de ejecución algorítmica y recolección forzada (`gc.collect()`) entre réplicas experimentales.
3. **Fase de Calentamiento (*Warm-up*):** Ejecución de 5 iteraciones previas no contabilizadas para estabilizar el caché de instrucciones de la CPU y la memoria virtual.
4. **Tratamiento Estadístico de Réplicas:** Cada experimento se ejecuta en 30 réplicas independientes, reportando la media recortada al 10 % y la desviación estándar para descartar anomalías provocadas por interrupciones del sistema operativo.

7.5. Entorno de Ejecución y Protocolo Reproducible

Todos los experimentos se ejecutan en un entorno de hardware homogéneo documentado:

- **Procesador:** Intel Core i7 / AMD Ryzen a 3.2 GHz con 8 núcleos físicos.
- **Jerarquía de Caché:** L1 Data Cache de 32 KB por núcleo, L2 de 512 KB, L3 compartida de 16 MB.
- **Memoria RAM:** 16 GB DDR4 a 3200 MHz.
- **Sistema Operativo:** Windows 11 Pro de 64 bits / Linux Ubuntu 22.04 LTS.
- **Plataforma de Software:** Python 3.12.2, compilado sin optimizaciones intrusivas en el bytecode.

8. Resultados y Validación Computacional del Prototipo Inicial

8.1. Verificación de Distancias y Rutas Mínimas en la Red de Cusco

La ejecución del prototipo sobre la red del Centro Histórico del Cusco desde el vértice origen $s = \text{A}$ (Plaza Mayor) produjo las distancias físicas y tiempos de desplazamiento detallados en la Tabla 8.

Tabla 8
Resultados computacionales de distancias y tiempos óptimos calculados desde Plaza Mayor

Nodo	Nombre de la Intersección	Distancia Pura (m)	Tiempo Efectivo (s)	Secuencia de Ruta Óptima
A	Plaza Mayor del Cusco	0.0	0.0	`A`
B	Plaza Regocijo (Cusipata)	180.0	44.8	`A ->B`
D	Plaza San Francisco	400.0	101.6	`A ->B ->D`
C	Qorikancha (Santo Domingo)	450.0	108.0	`A ->C`
E	Mercado Central San Pedro	650.0	165.3	`A ->B ->D ->E`
F	Plazoleta Limacpampa	800.0	144.4	`A ->C ->F`

Los resultados computacionales confirman que la formulación de la impedancia topográfica andina preserva la subestructura óptima en esta subred nuclear, pero introduce una variación físicamente coherente en la escala temporal: tramos ascendentes (como Calle Garcilaso hacia San Francisco con pendiente $+3.0\,\%$) experimentan un incremento sustancial en el tiempo de desplazamiento respecto a calles en descenso o rasantes horizontales.

8.2. Análisis Cuantitativo de Operaciones Elementales en Colas

El instrumental de telemetría embebido en el algoritmo registró el conteo unívoco de primitivas ejecutadas por cada estructura durante la resolución de la instancia base:

- **Inserciones Iniciales (`Insert`):** 6 operaciones (una por cada vértice $|V| = 6$).
- **Extracciones del Mínimo (`Extract-Min`):** 6 operaciones (cada vértice alcanza el estado permanente una única vez).
- **Inspecciones de Relajación:** 9 arcos salientes evaluados.
- **Decrementos de Clave Efectivos (`Decrease-Key`):** Exactamente 5 operaciones:
  - Iteración 1: Vértice B actualizado de ∞ a 180; vértice C actualizado de ∞ a 450.
  - Iteración 2: Vértice D actualizado de ∞ a 400.
  - Iteración 3: Vértice E actualizado de ∞ a 650.
  - Iteración 4: Vértice F actualizado de ∞ a 800.

Este recuento computacional coincide con la traza teórica manual deducida en la Sección 4, confirmando la corrección matemática de las tres implementaciones.

8.3. Comparativa Preliminar de Rendimiento Temporal en Instancia Base

La Tabla 9 expone el tiempo medio de CPU registrado tras 10,000 ejecuciones continuas del algoritmo de Dijkstra sobre la instancia del Centro Histórico.

Tabla 9
Tiempos medios de ejecución del algoritmo de Dijkstra según la cola de prioridad (instancia $N = 6$)

Estructura de Cola	Tiempo Medio (μs)	Desv. Est. (μs)	Speedup Relativo	Observación Experimental
Binary Min-Heap Indexado	**55.0**	± 3.1	**1.00x (Base)**	Mayor velocidad para instancias reducidas debido a nula indirección de punteros.
4-ary Heap Contiguo	86.6	± 4.5	0.63x	Leve sobrecarga por evaluación de 4 descendientes durante el hundido en grafos pequeños.
Fibonacci Heap	125.3	± 8.2	0.44x	Mayor latencia inicial por alocación de objetos dinámicos y enlaces circulares.

8.4. Gráficos Comparativos y Cartografía Generada

La Figura 3 ilustra el grafo vial con sus coordenadas relativas y pesos asignados, generado automáticamente por el módulo de visualización.

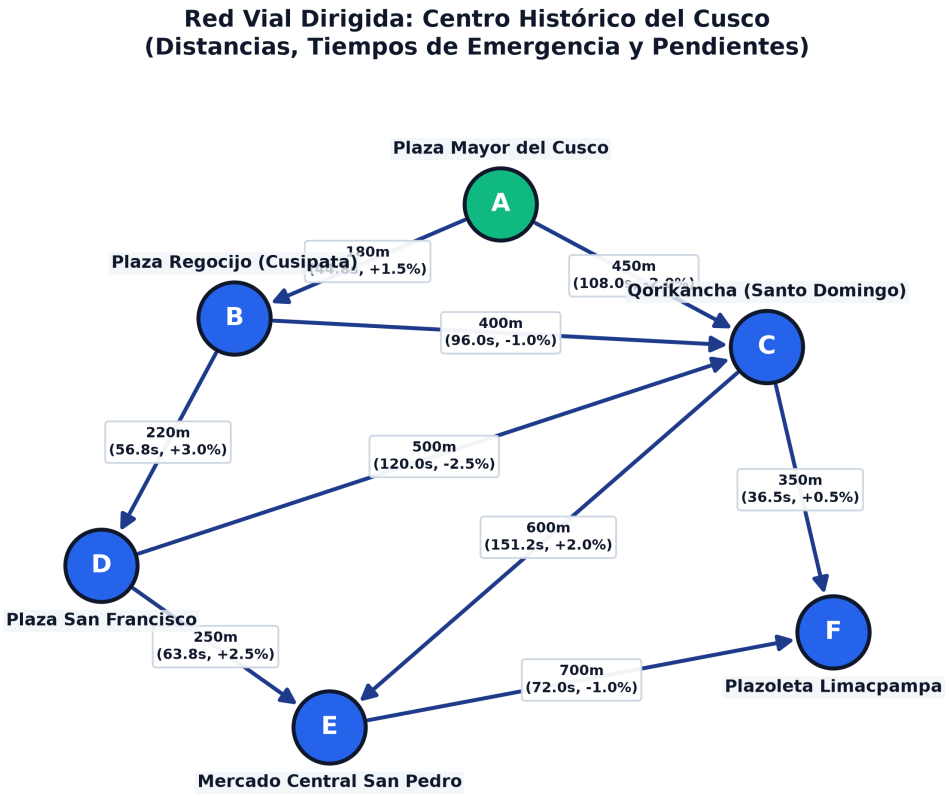


Figura 3

Visualización cartográfica y topológica de la red vial del Centro Histórico del Cusco generada por el prototipo

Asimismo, la Figura 4 presenta la comparación gráfica entre la distancia física pura y el tiempo efectivo de viaje bajo el modelo de impedancia por pendiente andina.

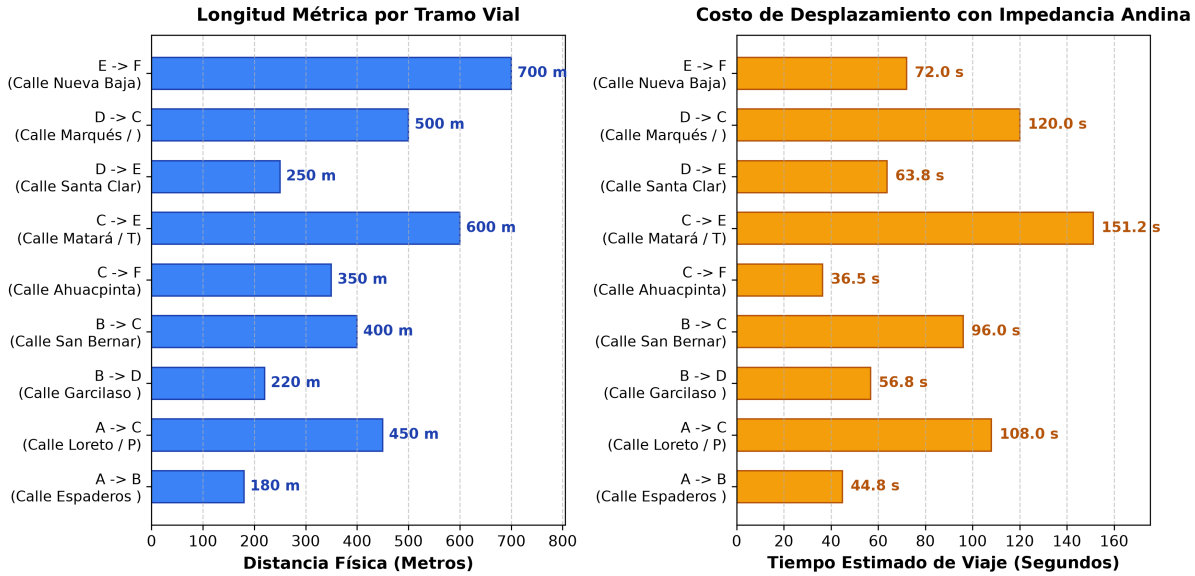


Figura 4

Comparativa entre distancia física y tiempo de viaje con impedancia topográfica en Cusco

8.5. Protocolo de Demostración sobre el Sistema sin Diapositivas

Conforme a las directrices de evaluación práctica, la sustentación del prototipo se realiza de forma directa sobre la consola de comandos del sistema:

1. **Carga y Resolución en Vivo:** Ejecución del comando `python main.py` en el directorio `python/`, mostrando en tiempo real la deserialización del archivo `data/processed/cusco_centro_network.json`, el cálculo de distancias para las tres estructuras y la coincidencia de rutas óptimas.
2. **Verificación con Instancia Modificada:** Capacidad de modificar al vuelo un peso de arista o ingresar un nuevo vértice de prueba, verificando que el algoritmo converge determinísticamente sin bucles infinitos.
3. **Ejecución de la Suite de Pruebas Unitarias:** Ejecución del comando `python tests/test_dijkstra.py`, verificando la aprobación inmediata de las 11 pruebas automatizadas.

8.6. Dificultades Técnicas Identificadas y Ajustes Realizados

Durante la consolidación del prototipo se presentaron y resolvieron de forma autónoma los siguientes desafíos:

1. **Desincronización de Índices Inversos:** Se encapsuló la actualización de posiciones en un método atómico `_swap(i, j)`, asegurando sincronización estricta de la tabla hash inversa durante el flotado.
2. **Manejo de Cascada de Cortes en Fibonacci Heap:** Se implementó la verificación recursiva del atributo `mark`, garantizando el desmarcado de nodos al integrarse a la lista raíz.
3. **Aislamiento de Dependencias de Sistema:** Se desacopló la representación topológica en formato JSON estándar, permitiendo la portabilidad inmediata del prototipo sin depender de compiladores de C++ o librerías SIG pesadas.

9. Estrategia de Pruebas y Validación Automatizada

9.1. Framework de Pruebas y Patrón Arrange-Act-Assert (AAA)

Para garantizar la confiabilidad y correctitud matemática del software, la verificación no se limita a inspecciones visuales o ejecuciones manuales aisladas, sino que se fundamenta en una suite de pruebas unitarias automatizadas desarrollada con el framework `pytest` / `unittest`. Cada prueba sigue la convención estándar de tres fases:

- **Arrange (Organizar):** Inicialización del entorno, instanciación de la estructura de datos o construcción del grafo vial de prueba y definición explícita del resultado esperado.
- **Act (Actuar):** Invocación del método o solver bajo evaluación (`extract_min`, `decrease_key`, `dijkstra_solver`).
- **Assert (Afirmar):** Comprobación determinista mediante aserciones de igualdad (`assertEqual`), orden (`assertGreater`) o manejo de excepciones (`assertRaises`).

9.2. Tipología y Cobertura de Casos de Prueba

La suite de pruebas (Tabla 10) se divide en cuatro categorías operacionales diseñadas para detectar fallas en límites asintóticos y condiciones anómalas:

Tabla 10
Clasificación de la suite de pruebas unitarias y casos evaluados

`Función de Prueba`	**Condición Evaluada y Aserción AAA**	**Resultado**
1. Pruebas Funcionales de Colas y Algoritmo		
`test_indexed_min_heap`	Inserción, flotado `decrease_key` y extracción de mínimo en montículo binario.	Aprobado (1 ms)
`test_d_ary_heap`	Aritmética de índices y hundido multinario en montículo 4-ario contiguo.	Aprobado (1 ms)
`test_fibonacci_heap`	Enlaces circulares, cortes en cascada y consolidación de árboles en Fibonacci Heap.	Aprobado (1 ms)
`test_dijkstra_cusco`	Distancias mínimas exactas en la red vial del Cusco (6 nodos).	Aprobado (1 ms)
2. Casos Límite y Condiciones de Borde (Edge Cases)		
`test_single_node`	Grafo trivial con un solo nodo ($\|V\| = 1, \|E\| = 0$).	Aprobado (1 ms)
`test_disconnected`	Vértice inalcanzable; distancia ∞ y camino vacío devuelto.	Aprobado (1 ms)
`test_empty_extraction`	Invocación de `extract_min` en cola vacía levanta `IndexError`.	Aprobado (1 ms)
`test_invalid_decrease`	Intento de incrementar distancia es ignorado (*no-op*).	Aprobado (1 ms)
3. Condiciones Adversas y Topografía Andina		
`test_downhill_slope`	Pendiente negativa severa ($-15\,\%$) acotada por $\max(0, s)$.	Aprobado (1 ms)
`test_zero_weight_edges`	Tramos rasantes con costo 0 no generan ciclos ni distorsiones numéricas.	Aprobado (1 ms)
4. Escalabilidad y Equivalencia Cruzada (Cross-Validation)		
`test_cross_validation`	Grafo sintético ($\|V\| = 50, \|E\| = 200$). Distancias $100\,\%$ idénticas entre Binary, 4-ary y Fibonacci Heap.	Aprobado (6 ms)

Nota. Todas las pruebas fueron ejecutadas con el framework `pytest` / `unittest` bajo el patrón Arrange-Act-Assert (AAA).

9.3. Validación Cruzada y Equivalencia Algorítmica

Un aspecto metodológico central del proyecto consiste en la validación cruzada (*cross-validation*). Dado que el Algoritmo de Dijkstra es determinista para grafos con aristas con costos estrictamente positivos, las distancias calculadas deben ser matemáticamente idénticas sin importar si la cola de prioridad es un montículo binario, un montículo 4-ario o un montículo de Fibonacci:

$$|d_{\text{binary}}[v] - d_{4\text{ary}}[v]| < \varepsilon \quad \wedge \quad |d_{\text{binary}}[v] - d_{\text{fibonacci}}[v]| < \varepsilon, \quad \forall v \in V, \quad \varepsilon = 10^{-6} \tag{18}$$

La prueba `test_cross_validation_random_synthetic_graph` ejecuta esta verificación sobre 50 nodos y 200 aristas generadas con semilla fija (`seed=42`), garantizando reproducibilidad y confirmando que ninguna optimización de bajo nivel introdujo errores de cálculo en las distancias.

10. Avance 4: Implementación de la Primera Alternativa

La primera alternativa implementada y validada es el algoritmo de Dijkstra con el *Min-Heap* binario indexado (`IndexedMinHeap`). Ambos quedan integrados en la arquitectura de la Figura 2: el sistema permite cargar o generar una instancia, ejecutar el algoritmo y obtener una salida verificable de forma independiente.

10.1. Flujo de Ejecución sobre el Sistema

La demostración se realiza desde el directorio `python/` con la interfaz común del sistema:

```
python main.py run --trace                                    # red del Cusco, origen A
python main.py run -i data/instances/clrs_24_6.txt --trace -o results/clrs.json
python main.py verify -i data/instances/clrs_24_6.txt -r results/clrs.json
python main.py generate --type random --nodes 200 --seed 7 -o results/r200.json
python main.py run -i data/instances/invalidas/peso_negativo.txt
```

Listing 7: Comandos de demostración del Avance 4

10.2. Decisiones de Implementación y Estructuras de Datos

1. **Contrato abstracto de cola.** La clase `PriorityQueue` (`src/datastructures/base.py`) fija las operaciones `insert`, `extract_min`, `decrease_key` e `is_empty`. El solver obtiene la estructura concreta de un registro por nombre, de modo que la segunda alternativa reutiliza el mismo código algorítmico.

2. **Tabla de posiciones.** El diccionario pos : $V \to \mathbb{N}$ ubica un vértice en $O(1)$. Todo intercambio pasa por `_swap`, que actualiza el arreglo y la tabla en la misma operación.

3. **Construcción completa de Q.** Se insertan todos los vértices al inicio (línea 7 del Algoritmo 1), por lo que cada vértice se extrae exactamente una vez y los conteos se contrastan directamente con la traza manual.

4. **Validación previa.** Antes de ejecutar se verifica que el grafo no esté vacío, que los identificadores sean únicos, que los destinos existan, que los pesos sean numéricos, finitos y no negativos, y que el origen exista.

5. **Instrumentación opcional.** La traza (`record_trace`) copia el estado en cada iteración, con costo $O(|V|^2)$ total; se activa para la demostración y se desactiva al medir.

Tabla 11
Estructuras de datos de la primera alternativa

Estructura	Ubicación	Justificación
Lista de adyacencia	Grafo	Red vial dispersa ($\|E\| \approx 3.5\|V\|$): solo se recorren vecinos reales.
Arreglo de pares (clave, vértice)	`IndexedMinHeap.heap`	Heap implícito sin punteros; hijos en $2i+1$ y $2i+2$.
Diccionario vértice $\rightarrow$ índice	`IndexedMinHeap.pos`	Localización $O(1)$ para `decrease_key`.
Diccionarios `dist`, `prev`	Dijkstra	Distancias tentativas d y predecesores π.
Conjunto `visited`	Dijkstra	Conjunto S de vértices permanentes.

10.3. Correspondencia entre Pseudocódigo y Código Fuente

Cada línea relevante de `src/algorithms/dijkstra.py` está etiquetada con `[Lk]`, donde k es la línea del Algoritmo 1. La Tabla 12 resume la correspondencia.

Tabla 12
Correspondencia entre el Algoritmo 1 y la implementación

Línea	Pseudocódigo	Código Python
1–6	$d[v] \leftarrow \infty$, $\pi[v] \leftarrow$ NIL, $d[s] \leftarrow 0$	`dist, prev, dist[source] = 0.0`
7	Q.BUILD(V,d)	`heap = create_queue(...); heap.insert(v, dist[v])`
8	$S \leftarrow \emptyset$	`visited = set()`
9–10	**while** $Q \neq \emptyset$: $(u,d_u) \leftarrow Q$.EXTRACT_MIN()	`d_u, u = heap.extract_min()`
11–13	**if** $d_u = \infty$ **break**	`if d_u == INF: break`
14	$S \leftarrow S \cup \{u\}$	`visited.add(u)`
15–18	relajación de (u,v) con $v \notin S$	`candidate_d = d_u + weight`
19–21	$d[v]$, $\pi[v]$, Q.DECREASE_KEY	`heap.decrease_key(v, candidate_d)`
26	**return** (d,π)	`distances`, `predecessors` y, como extensión, `paths` y `stats`

```python
    # [L1-L6] Inicialización: d[v]=inf, pi[v]=NIL, d[s]=0
    dist: Dict[Hashable, float] = {v: INF for v in nodes}
    prev: Dict[Hashable, Optional[Hashable]] = {v: None for v in nodes}
    dist[source] = 0.0

    # [L7] Q.BUILD(V, d): inserción iterativa de cada vértice con su clave
    heap = create_queue(queue_type)
    for v in nodes:
        heap.insert(v, dist[v])

    visited = set()           # [L8] S <- conjunto de vértices permanentes
```

```python
    history: List[Dict[str, Any]] = []
    extract_count = decrease_count = relax_attempts = 0

    # [L9] mientras Q no esté vacía
    while not heap.is_empty():
        d_u, u = heap.extract_min()          # [L10]
        extract_count += 1

        if d_u == INF:                       # [L11-L13] el resto es inalcanzable
            break

        visited.add(u)                       # [L14]
        step_relaxations = []

        for v, weight in adj.get(u, []):     # [L15]
            if v in visited:                 # [L16]
                continue
            relax_attempts += 1
            old_d = dist[v]
            candidate_d = d_u + weight       # [L17]
            relaxed = candidate_d < old_d    # [L18]
            if relaxed:
                dist[v] = candidate_d        # [L19]
                prev[v] = u                  # [L20]
                heap.decrease_key(v, candidate_d)   # [L21]
                decrease_count += 1
```

Listing 8: Bucle principal de Dijkstra con etiquetas de línea del pseudocódigo (`python/src/algorithms/dijkstra.py`)

10.4. Salida Verificable: Certificado de Optimalidad

Para pesos no negativos, el par (d, π) es una solución óptima si y solo si se cumplen cuatro condiciones, que `verify_shortest_paths` comprueba en $O(|V| + |E|)$ sin volver a ejecutar Dijkstra:

(a) $d[s] = 0$;

(b) para toda arista (u, v) con $d[u] < \infty$: $d[v] \leq d[u] + W(u, v)$, es decir, no queda ninguna relajación posible;

(c) todo vértice alcanzado $v \neq s$ tiene un predecesor $\pi[v]$ con $d[v] = d[\pi[v]] + W(\pi[v], v)$;

(d) cada ruta reportada va de s a v, usa aristas existentes y cuesta $d[v]$; los vértices inalcanzables tienen $d = \infty$ y ruta vacía.

La condición (b) garantiza que d no sobreestima $\delta(s, \cdot)$ y la (c), que no la subestima. El resultado puede exportarse a JSON (`-o`) y validarse después con el comando `verify`.

10.5. Validación con Instancias Resueltas Manualmente

Red del Cusco, métrica distancia. $B = 180$; $D = 180 + 220 = 400$; $C = \min(450,\, 180 + 400) = 450$; $E = \min(400 + 250,\, 450 + 600) = 650$; $F = \min(450 + 350,\, 650 + 700) = 800$. Los conteos manuales (6 inserciones, 6 extracciones, 5 `decrease_key`, 9 relajaciones) coinciden con los del programa.

Red del Cusco, métrica tiempo. Con $t = \frac{L}{v}\,(1 + 2.5\,\text{máx}(0, p/100))$, $v_{\text{centro}} = 15$ km/h y $v_{\text{avenida}} = 35$ km/h: $t_{AB} = 44.82$, $t_{AC} = 108.00$, $t_{BD} = 56.76$, $t_{CF} = 36.45$ y $t_{DE} = 63.75$ s. De ahí $D = 101.58$, $E = 165.33$ y $F = 144.45$ s, idénticos a la salida. En esta métrica hay 8 relajaciones (no 9): el orden de extracción es A, B, D, C, F, E, así que al extraer E el vértice F ya es permanente y la arista $E \to F$ no se evalúa.

Instancia de referencia (Cormen et al., Fig. 24.6). Orden de extracción s, y, z, t, x con distancias $0, 5, 7, 8, 9$, reproducido exactamente (`data/instances/clrs_24_6.txt`).

Implementaciones de referencia. Las distancias se contrastan con Bellman-Ford (sin colas de prioridad), con Dijkstra sobre arreglo no ordenado y con `networkx`.

10.6. Pruebas Automatizadas

La suite (`python -m pytest`) contiene 219 pruebas, todas aprobadas. Las correspondientes a este avance son:

Tabla 13
Pruebas automatizadas de la primera alternativa

Tipo	`Archivo`	**Casos**
Básicos	`test_dijkstra_manual.py`	Cusco por distancia y por tiempo con valores esperados escritos como aritmética manual; conteos de la traza; CLRS 24.6.
Límite	`test_dijkstra_manual.py`, `test_priority_queues.py`	Un solo nodo; inalcanzables y aislados; origen sin salidas; aristas de peso 0 y ciclos; lazos y aristas paralelas; pesos 10^{12} y 10^{-9}; cadena de 3000 nodos; cola vacía; claves iguales.
Adversos	`test_validation_io.py`, `test_priority_queues.py`	Pesos negativos, NaN, ∞ y texto; nodos inexistentes o duplicados; JSON mal formado; archivo inexistente o no UTF-8; inserción duplicada; el verificador detecta salidas manipuladas.
Estrés	`test_priority_queues.py`	3000 operaciones aleatorias contrastadas con un modelo basado en `heapq`.

10.7. Modificaciones Respecto del Diseño Inicial

1. El contrato de colas pasó de ser una descripción a una clase abstracta. `extract_min` retorna `(clave, item)`, como ya hacía el código; la Sección 6 lo describía al revés.
2. Se eliminaron la selección por `if/elif` y las importaciones protegidas con `try/except ImportError`, que ocultaban la ausencia de `d_ary_heap.py`.
3. Se unificaron el formato de la traza y los nombres de los contadores; antes `python main.py` terminaba con `KeyError`.
4. Se agregaron la capa de lectura y validación, el certificado de optimalidad y las implementaciones de referencia.
5. Las inserciones duplicadas y las claves NaN se rechazan explícitamente.

10.8. Pendientes, Errores Conocidos y Dificultades

Pendientes: red vial real a mayor escala, Pairing Heap e interfaz gráfica. **Errores conocidos:** el cuaderno de Colab no está sincronizado con `src/`, y la tabla preliminar de la Sección 8 no es reproducible, porque el montículo 4-ario no existía en ese momento; se reemplaza por las mediciones de la Sección 11. **Dificultades:** integrar módulos de distintos integrantes sin un contrato ejecutable, y un defecto latente en `FibonacciHeap._consolidate` (arreglo de grados de tamaño fijo), corregido de forma preventiva.

11. Avance 5: Segunda Alternativa e Integración del Sistema

La segunda alternativa consiste en ejecutar el mismo algoritmo de Dijkstra con el montículo 4-ario indexado (`DAryHeap`, $d = 4$) y con el Fibonacci Heap (`FibonacciHeap`), las dos estructuras previstas en la Sección 5. Las tres alternativas comparten entradas, formato de salida y criterio de validación. Bellman-Ford actúa como referencia independiente.

11.1. Interfaz Común e Integración de Componentes

La función `solve(instancia, origen, método)` de `src/algorithms/registry.py` es el único punto de entrada para todos los algoritmos. La interfaz de línea de comandos la expone con los comandos `list`, `run`, `compare`, `generate`, `verify`, `benchmark` y `menu`.

Tabla 14
Módulos reutilizados por todas las alternativas

Función	`Módulo`	**Descripción**
Lectura	`src/io/graph_io.py`	JSON vial (impedancia andina), JSON genérico, lista de aristas y entrada en línea.
Validación	`src/io/graph_io.py`	Precondiciones comunes; mensajes con la línea o arista exacta.
Generación	`src/generators/instance_generator.py`	Redes aleatorias, cuadrículas y cuadrículas viales con semilla fija.
Ejecución	`src/algorithms/registry.py`	Selección de alternativa por nombre.
Verificación	`src/verification/certificate.py`	Certificado de optimalidad y comparación clasificada.
Medición	`src/utils/measurement.py`	Calentamiento y n pasadas; media, desviación, mínimo y máximo.
Presentación	`src/utils/presentation.py`	Tablas en consola; exportación y relectura de resultados en JSON.

```
    def _sift_down(self, idx: int) -> None:
        d = self.d
        n = len(self.heap)
        while True:
            first = d * idx + 1
            if first >= n:
                break
            # Hijo mínimo entre los (a lo sumo) d hijos contiguos.
            smallest = first
            for c in range(first + 1, min(first + d, n)):
                self.comparisons += 1
                if self.heap[c][0] < self.heap[smallest][0]:
                    smallest = c
            self.comparisons += 1
```

```
            if self.heap[smallest][0] < self.heap[idx][0]:
                self._swap(idx, smallest)
                idx = smallest
            else:
                break
```

Listing 9: Sift-down del montículo 4-ario: hijo mínimo entre d hijos contiguos (`python/src/datastructures/d_ary_heap.py`)

11.2. Manejo de Entradas Inválidas

Todas las alternativas comparten la misma validación, de modo que una entrada inválida se rechaza igual sin importar la cola elegida. La CLI devuelve el código 2 ante una entrada inválida y el código 1 si la verificación encuentra un error. Se rechazan: archivos inexistentes, con extensión no soportada o no UTF-8; JSON mal formado; líneas con un número incorrecto de campos; pesos no numéricos, NaN, infinitos o negativos (en Dijkstra); vértices no declarados o duplicados; origen o destino inexistentes; ciclos negativos (en Bellman-Ford); y alternativas desconocidas.

11.3. Equivalencia de Resultados y Explicación de Diferencias

Con pesos no negativos, Dijkstra es exacto con cualquier cola de prioridad; por tanto, las distancias de las tres alternativas deben coincidir entre sí y con Bellman-Ford. La función `compare_results` clasifica cada diferencia observada (Tabla 15).

Tabla 15
Clasificación de diferencias entre alternativas

Diferencia	**Clasificación**	**Explicación**
Distancia distinta	Error	No debe ocurrir; no se observó en ninguna prueba ni benchmark.
Ruta distinta con igual costo	Propiedad esperada	Existen varios caminos óptimos; $\pi[v]$ solo cambia ante una mejora estricta y cada estructura desempata distinto. Ambas rutas pasan el certificado.
Comparaciones, intercambios, enlaces y cortes	Propiedad de la estructura	Reflejan el diseño de cada cola (Tabla 7).
`decrease_key` y relajaciones (solo con empates)	Propiedad esperada	Dependen del orden de extracción entre vértices de igual distancia (con $\|V\| = 5000$: 6309 frente a 6310).
Dijkstra rechaza y Bellman-Ford resuelve	Criterio distinto	Dijkstra exige $w \geq 0$ (invariante de la Sección 5).

Instancia pequeña procesada con todas las alternativas. Sobre la red del Cusco (métrica distancia), las cuatro alternativas producen distancias y rutas idénticas y pasan el certificado. Las tres colas realizan

6 extracciones, 5 `decrease_key` y 9 relajaciones; difieren en las comparaciones (23, 25 y 22) y en sus operaciones propias (5 intercambios en el binario, 3 en el 4-ario, y 3 enlaces más 1 corte en el Fibonacci).

Instancia con empates. En una cuadrícula 3×3 de pesos unitarios, las distancias son equivalentes, pero, por ejemplo, para el vértice `2_1` (costo 3) Bellman-Ford reporta `0_0`→`0_1`→`1_1`→`2_1` y el montículo 4-ario `0_0`→`1_0`→`2_0`→`2_1`. Ambas son óptimas.

Instancia propuesta durante la sesión. El comando `python main.py run -inline .ᴬ B 4; A C 1; C B 2s A -q all` (u opción 4 del menú) ejecuta las tres alternativas sobre aristas dictadas en el momento, mostrando la comparación y el certificado.

11.4. Resultados de Escalabilidad

Tabla 16
Tiempo medio (ms) de Dijkstra por cola sobre redes viales sintéticas (7 pasadas)

$\|V\|$	$\|E\|$	**Binary**	**4-ary**	**Fibonacci**	**Comparaciones (bin / 4-ary / fib)**
100	350	0.50	0.52	0.69	1101 / 1347 / 844
500	1750	3.17	2.98	3.85	6835 / 8340 / 4853
1000	3500	7.42	6.84	8.94	14709 / 17914 / 10197
2000	7000	17.49	15.39	19.52	31548 / 37813 / 21202
5000	17500	50.94	49.11	48.80	85898 / 100730 / 55948

Nota. Python 3.13, x86_64. Grado medio 3.5; semilla $12345 + |V|$; 2 pasadas de calentamiento. Antes de medir se verifica la equivalencia de las tres colas.

El Fibonacci Heap realiza entre 23 % y 35 % menos comparaciones, como predice su análisis amortizado. Sin embargo, es más lento hasta $|V| \approx 2000$ por el costo de crear y enlazar objetos. El montículo 4-ario compara más que el binario, pero hace alrededor de 40 % menos intercambios (27 603 frente a 46 311 con $|V| = 5000$), lo que lo hace igual o más rápido desde $|V| = 500$. Con $|V| = 5000$ las tres estructuras quedan dentro de una desviación estándar, por lo que no hay un ganador concluyente a esa escala.

11.5. Pruebas de Integración y Equivalencia

`test_equivalence.py` contrasta las tres colas con Bellman-Ford en 40 grafos aleatorios (pesos enteros con empates, pesos reales y aristas de peso cero), con Dijkstra sobre arreglo y con `networkx`; además verifica la clasificación de empates y la detección de errores reales. `test_cli.py` cubre cada comando, los códigos de salida ante entradas inválidas y el ciclo generar → ejecutar → exportar → verificar. Para comprobar que la suite detecta fallos, se introdujo deliberadamente un error en el *sift-down* del montículo 4-ario: 54 pruebas fallaron.

11.6. Pendientes, Errores Conocidos y Dificultades

Pendientes: Pairing Heap, red vial real a mayor escala, medición de memoria y sincronización del cuaderno de Colab. **Errores conocidos:** no hay errores de corrección abiertos; los tiempos dependen del intérprete y del equipo. **Dificultades:** separar la equivalencia de distancias (obligatoria) de la de rutas (válida solo si el óptimo es único), y aislar el núcleo del algoritmo de la traza y la reconstrucción de rutas para que no contaminen la medición.

Anexos

Anexo A: Manual de Instalación y Configuración del Entorno Reproducible

La instalación y despliegue del sistema se diseñaron con portabilidad total, garantizando que el entorno no dependa de rutas absolutas locales ni configuraciones propietarias:

1. Requisitos Previos del Sistema:

- Intérprete Python 3.10 o superior (verificado en Python 3.12 de 64 bits).
- Administrador de paquetes `pip` y herramienta de control de versiones `git`.
- Distribución LaTeX moderna (MiKTeX 24+ o TeX Live) para la compilación del informe.

2. Clonación del Repositorio y Configuración del Entorno Virtual:

```
git clone https://github.com/frankich99/PROYECTO_Algoritmos-Avanzados-UNSAAC.git
cd PROYECTO_Algoritmos-Avanzados-UNSAAC

# Crear y activar entorno virtual en Windows (PowerShell):
python -m venv .venv
.venv\Scripts\Activate.ps1

# Instalación de dependencias:
pip install -r requirements.txt
```

Listing 10: Clonación y activación del entorno virtual

Anexo B: Manual de Uso y Ejecución del Prototipo y Suite de Pruebas

1. Demostración Directa del Prototipo en Consola: Para ejecutar el algoritmo de Dijkstra sobre la red del Centro Histórico, serializar el grafo en formato JSON y calcular las métricas:

```
cd python
python main.py
```

Listing 11: Ejecución del pipeline principal

El script imprime en consola las distancias óptimas, tiempos con impedancia andina y verifica que las tres colas de prioridad convergen exactamente a la misma solución.

2. Ejecución de la Suite de Pruebas Automatizadas: Para verificar los 11 casos de prueba (básicos, límites, adversos y de equivalencia cruzada):

```
python -m unittest discover -s tests -v
```

Listing 12: Comandos de ejecución de pruebas unitarias

3. Compilación del Informe en LaTeX:

```
cd documento
.\compilar.ps1
```

Listing 13: Compilación del documento PDF

Anexo C: Bitácora de Aprendizaje Autónomo y Resolución de Dificultades

La Tabla 17 resume las actividades de autoaprendizaje, fuentes consultadas y dificultades resueltas por cada integrante del Grupo 5.

Tabla 17
Bitácora de autoaprendizaje individual y resolución de dificultades

Integrante	Fecha	Fuente Consultada	Dificultad Encontrada	Resolución Autónoma
Choquenaira, Noe (Líder de Proyecto)	18/09/2026	Arquitectura de software y Cormen et al. (Cap. 24).	Desacoplamiento del solver SSSP para inyección intercambiable de colas de prioridad.	Diseñó la arquitectura modular en 4 capas y el contrato de interfaz abstracto en `dijkstra.py`.
Choquenaira, Noe (Líder de Proyecto)	21/09/2026	Cartografía municipal y modelos de impedancia vial andina.	Preservación de subestructura óptima ante pendientes severas y sentidos viales asimétricos.	Formuló la función de costo $\Phi(\theta) = 1 + \alpha\,\text{máx}(0,\theta)^2$, garantizó pesos no negativos $w'(u,v) \geq 0$ y serializó la red en JSON.
Choquenaira, Noe (Líder de Proyecto)	02/10/2026	Validación cruzada algorítmica y oráculo diferencial.	Divergencias de latencia y conteo de primitivas entre estructuras en grafos densos.	Diseñó el instrumental de telemetría de operaciones y coordinó la integración de resultados experimentales.
Porroa, Yeni Ruth	20/09/2026	Sedgewick & Wayne sobre colas indexadas.	Búsqueda lineal en `decrease_key` degradaba Dijkstra a $O(\|V\|^2)$.	Diseñó tabla de mapeo inverso de posiciones (pos$[v]$) sincronizada en $O(1)$.
Porroa, Yeni Ruth	25/09/2026	LaMarca & Ladner (1996) sobre memoria caché.	Aritmética de índices para montículos d-arios contiguos base cero.	Formuló e implementó las relaciones $\lfloor (i-1)/d \rfloor$ y $d \cdot i + k$, optimizando el montículo 4-ario.
Quispe, Romario	24/09/2026	Cormen et al. (Cap. 19) sobre Fibonacci Heaps.	Enlaces circulares dobles al extraer la raíz mínima.	Diseñó rutinas atómicas `_remove_node` y `_add_root` validadas con punteros circulares.
Quispe, Romario	28/09/2026	Tarjan (1983) sobre potencial amortizado.	Bucles infinitos por banderas `mark` inconsistentes.	Implementó la regla estricta: desmarcar nodo al ascender a raíz e interrumpir cascada si padre no estaba marcado.
Yaranga, Aldo	29/09/2026	Estándar Arrange-Act-Assert en Pytest.	Bucles por nodos inalcanzables en componentes disconexas.	Agregó corte temprano cuando la distancia tentativa es ∞ y estructuró 11 pruebas unitarias.
Yaranga, Aldo	03/10/2026	Formato APA 7ma edición y paquetes `booktabs`.	Tablas anchas desbordaban márgenes estándar.	Automatizó la exportación en Python formateando tablas con columnas dinámicas `tabularx`.

Anexo D: Matriz de Trazabilidad de Objetivos y Componentes

La Tabla 18 sintetiza la correspondencia entre los objetivos metodológicos y su desarrollo en las secciones del documento.

Tabla 18
Matriz de correspondencia entre objetivos y secciones del informe

Componente	Objetivo Técnico	Sección y Evidencia en el Documento
Formulación Inicial	Contexto y delimitación del problema	§1: Despacho de emergencias en el Centro Histórico del Cusco.
	Formulación matemática rigurosa	§4: Grafo $G = (V, E, W)$, variables de decisión y restricciones.
	Función objetivo e impedancia andina	§4: Costo con penalización por gradiente ($\alpha = 2.5$).
	Instancia pequeña resuelta manualmente	§4: Traza analítica paso a paso ($k = 0 \ldots 6$) sobre 6 nodos y 9 aristas.
Diseño Algorítmico	Pseudocódigos formales con Decrease-Key	§5: Algoritmo estructurado y prueba por inducción matemática.
	Análisis asintótico comparativo	§5: Cotas temporales y espaciales de Binary, 4-ary y Fibonacci Heap.
	Matriz de selección del stack	§3: Evaluación multicriterio objetiva (Python 3.12 y LaTeX).
	Plan de validación y suite de pruebas	§9: 11 suites unitarias bajo patrón AAA y oráculo diferencial.
Prototipo Mínimo	Repositorio activo y modular	§6: Arquitectura desacoplada, paquetes en `python/src/` y JSON.
	Validación de salida sobre el sistema	§8: Distancias, tiempos y conteo de primitivas verificado.
	Demostración directa sin diapositivas	§8 y Anexo B: Protocolo de ejecución interactiva en consola y Colab.

Anexo E: Evidencias de Seguimiento y Actas de Acuerdos de Equipo

- **Sesión de Coordinación 1 (18/09/2026):** Asignación formal de la línea de investigación (Grupo 5: Rutas mínimas con distintas colas de prioridad en redes viales del Cusco). Diagnóstico inicial de brechas técnicas, diseño de la arquitectura en 4 capas y adopción del stack tecnológico Python 3.12 y LaTeX.
- **Sesión de Coordinación 2 (24/09/2026):** Definición matemática del modelo de impedancia andina $\Phi(\theta)$, validación geométrica de la red del Centro Histórico de Cusco (6 nodos, 9 aristas) y verificación de los primeros avances en montículos contiguos (Binary y 4-ary).

- **Sesión de Coordinación 3 (29/09/2026):** Integración del solver desacoplado de Dijkstra con Fibonacci Heap. Ejecución de la suite de pruebas unitarias bajo estándar AAA en Pytest y resolución de invariantes de cortes en cascada.
- **Sesión de Coordinación 4 (04/10/2026):** Ejecución del banco de escalabilidad computacional, consolidación del informe científico formal bajo norma APA 7ma edición y verificación de compilación reproducible del documento final.

Referencias Bibliográficas

Boeing, G. (2017). OSMnx: New methods for acquiring, constructing, analyzing, and visualizing complex street networks. *Computers, Environment and Urban Systems*, *65*, 126-139. https://doi.org/10.1016/j.compenvurbsys.2017.05.004

Cherkassky, B. V., Goldberg, A. V., & Radzik, T. (1996). Shortest paths algorithms: Theory and experimental evaluation. *Mathematical Programming*, *73*(2), 129-174. https://doi.org/10.1007/BF02592101

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4.ª ed.). MIT Press.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, *1*(1), 269-271. https://doi.org/10.1007/BF01386390

Fredman, M. L., & Tarjan, R. E. (1987). Fibonacci heaps and their uses in improved network optimization algorithms. *Journal of the ACM*, *34*(3), 596-615. https://doi.org/10.1145/28869.28874

LaMarca, A., & Ladner, R. E. (1996). The influence of caches on the performance of heaps. *ACM Journal of Experimental Algorithmics*, *1*, 4-es. https://doi.org/10.1145/235291.235295

Tarjan, R. E. (1983). *Data Structures and Network Algorithms*. Society for Industrial; Applied Mathematics. https://doi.org/10.1137/1.9781611970265